



人工智能

陈科海

计算机科学与技术学院
哈尔滨工业大学（深圳）

chenkehai@hit.edu.cn



第3章 确定性推理

目 录

3.1 概述

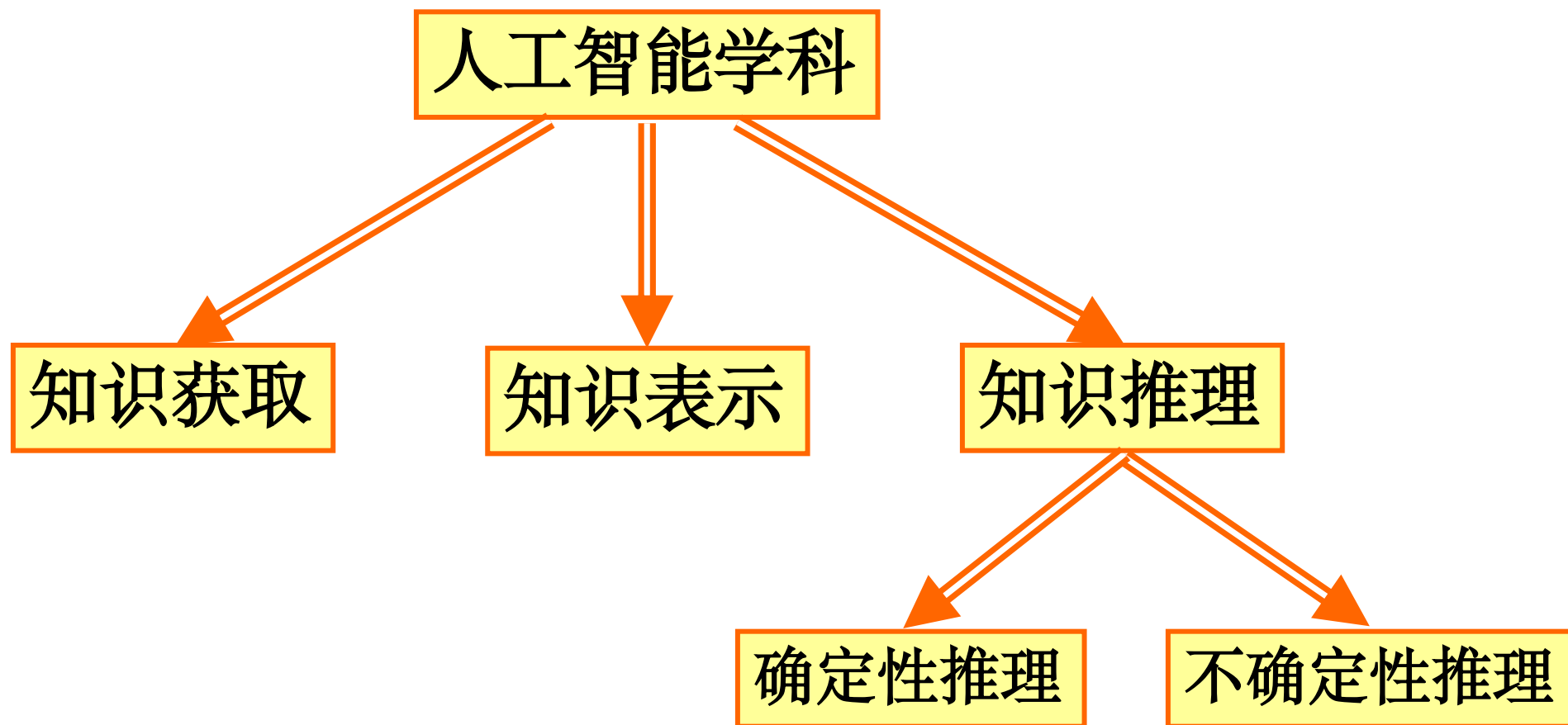
3.2 产生式系统

3.3 自然演绎推理

3.4 归结演绎推理

目 录

- **掌握几种不同的推理方法**
- **掌握一阶谓词逻辑基础、置换与合一的概念，掌握求取最一般合一置换的方法。**
- **掌握归结原理及归结推理方法。**
- **掌握利用归结原理进行定理证明的方法。**
- **掌握利用归结原理进行问题求解的方法。**
- **了解归结过程中的控制策略。**



3.1.1 推理概述

□ **推理的定义：按照某种策略从已有事实和知识推出结论的过程**

心理学对推理有两种解释：

➤ **从结构的角度**

推理由两个以上的判断所组成，把判断定义为对客观事物做出肯定或否定的思维活动；认为判断是在概念的基础上进行的，所揭示的是概念之间联系和关系。

3.1.1 推理概述

□ 推理的定义：按照某种策略从已有事实和知识推出结论的过程

心理学对推理有两种解释：

➤ 从结构的角度

推理由两个以上的判断所组成，把判断定义为对客观事物做出肯定或否定的思维活动；认为判断是在概念的基础上进行的，所揭示的是概念之间联系和关系。例如，若有以下两个判断：

① 计算机系的学生都会编程序；

② 程强是计算机系的一名学生；

则可得出下面第三个判断：

③ 程强会编程序。

因此，推理就是对已有判断进行分析和综合，再得出新的判断的过程。

3.1.1 推理概述

➤ 从过程的角度

认为推理是在给定信息和已有知识的基础上的一系列加工操作，提出了如下人类推理的公式：

$$y=F(x, k)$$

其中， x 为推理时给出的信息， k 为推理时可用的领域知识和特殊事例， F 为可用的一系列操作， y 为推理过程所得到的结论。

3.1.1 推理概述

➤ 从过程的角度

认为推理是在给定信息和已有知识的基础上的一系列加工操作，提出了如下人类推理的公式：

$$y=F(x, k)$$

其中， x 为推理时给出的信息， k 为推理时可用的领域知识和特殊事例， F 为可用的一系列操作， y 为推理过程所得到的结论。

➤ 推理的机器实现

人工智能中的推理是由推理机完成的。所谓推理机，是指系统中用来实现推理的那段程序。

根据推理所用知识的不同，推理方式和推理方法的不同，推理机的构造也有所不同。

3.1.2 推理的分类

□ 按推理的逻辑基础分类

➤ **演绎推理**：从已知的一般性知识出发，推理出适合于某种个别情况的结论过程。即从**一般到个别**的推理。

➤ **常用形式**：三段论法（大前提、小前提、结论）

❖ **大前提**：是已知的一般性知识或推理过程得到的判断；

❖ **小前提**：是关于某种具体情况或某个具体实例的判断；

❖ **结 论**：是由大前提推出的，并且适合于小前提的判断。

3.1.2 推理的分类

➤ 演绎推理实例

① 音乐系的学生至少会演奏一种乐器；（大前提）

② 李聪是音乐系的一名学生；（小前提）

3.1.2 推理的分类

➤ 演绎推理实例

① 音乐系的学生至少会演奏一种乐器；（大前提）

② 李聪是音乐系的一名学生；（小前提）

③ 李聪至少会演奏一种乐器。（结论）

3.1.2 推理的分类

□ 按推理的逻辑基础分类

➤ **归纳推理**：从大量特殊事例出发，归纳出一般性结论的推理过程。

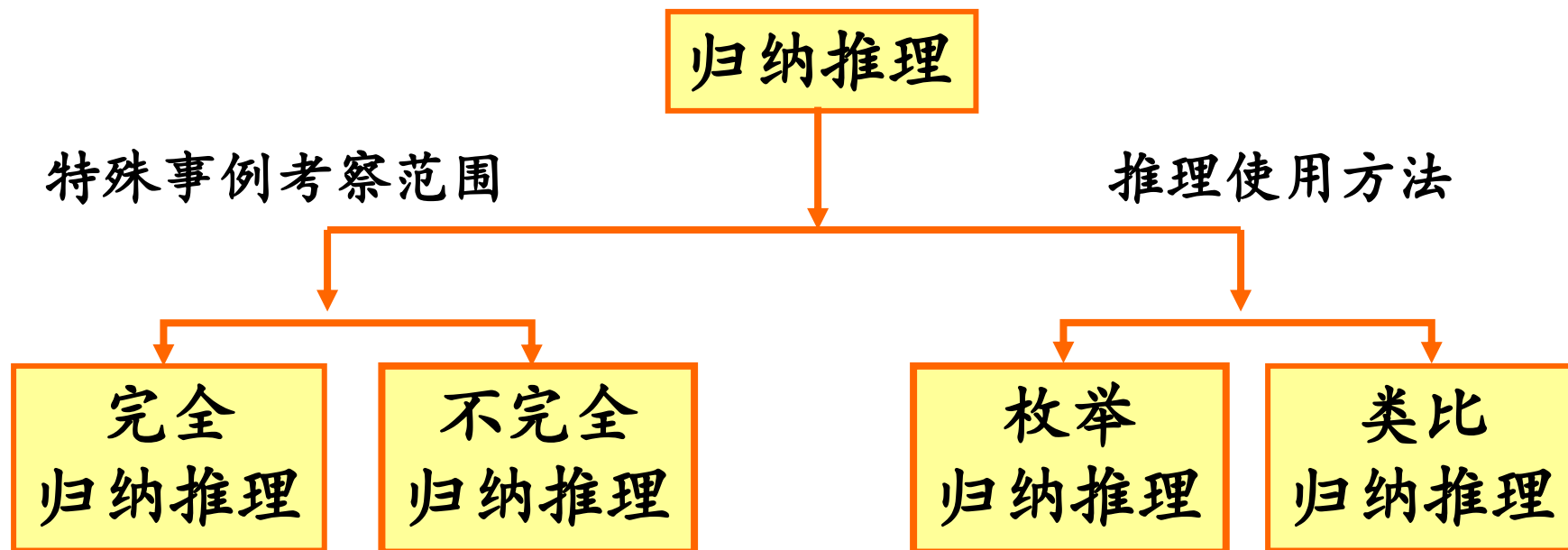
即从**个别到一般**的推理。

3.1.2 推理的分类

□ 按推理的逻辑基础分类

➤ **归纳推理**：从大量特殊事例出发，归纳出一般性结论的推理过程。

即从**个别到一般**的推理。



3.1.2 推理的分类

□ 按推理的逻辑基础分类

- **演绎推理**: 在已知领域内的一般性知识的前提下, 通过演绎求解一个具体问题或者证明一个结论的正确性。它所得出的结论实际上早已蕴含在一般性知识的前提中, 演绎推理只不过是已将已有事实揭露出来, 因此它不能增殖新知识。
- **归纳推理**: 所推出的结论是没有包含在前提内容中的。这种由个别事物或现象推出一般性知识的过程, 是增殖新知识的过程。

3.1.2 推理的分类

□ 按推理的逻辑基础分类

➤ **演绎推理**：在已知领域内的一般性知识的前提下，通过演绎求解一个具体问题或者证明一个结论的正确性。它所得出的结论实际上早已蕴含在一般性知识的前提中，演绎推理只不过是已将已有事实揭露出来，因此它不能增殖新知识。

➤ **归纳推理**：所推出的结论是没有包含在前提内容中的。这种由个别事物或现象推出一般性知识的过程，是增殖新知识的过程。

例如，一位计算机维修员，从书本知识，到通过大量实例积累经验，是一种归纳推理方式。运用这些一般性知识去维修计算机的过程则是演绎推理。

3.1.2 推理的分类

□ 按所用知识的确定性分类

- **确定性推理：**推理时所有用的知识和证据都是确定的，推出的结论也是确定的，其真值或者为真或者为假，没有第三种情况出现。
- **不确定性推理：**推理时所用的知识和证据不都是确定的，推出的结论也不确定的。

3.1.2 推理的分类

□ 按所用知识的确定性分类

- **确定性推理：**推理时所有用的知识和证据都是确定的，推出的结论也是确定的，其真值或者为真或者为假，没有第三种情况出现。
- **不确定性推理：**推理时所用的知识和证据不都是确定的，推出的结论也不确定的。

□ 按推理中所用知识是否具有启发性分类

- **启发式推理：**推理过程中应用与问题有关的启发性知识，即解决问题的策略、技巧及经验，以加快推理过程，提高搜索效率。
- **非启发式推理：**在推理过程中，不运用启发性知识，只按照一般的控制逻辑进行推理。这种方法缺乏对求解问题的针对性，所以推理效率较低，容易出现“组合爆炸”问题。

3.1.3 推理的控制策略及分类

□ 推理的控制策略

推理过程不仅依赖于所用的推理方法，同时也依赖于推理的控制策略。

推理的控制策略是指**如何使用领域知识使推理过程尽快达到目标的策略。**

3.1.3 推理的控制策略及分类

□ 推理的控制策略

推理过程不仅依赖于所用的推理方法，同时也依赖于推理的控制策略。

推理的控制策略是指**如何使用领域知识使推理过程尽快达到目标的策略**。

□ 控制策略的分类

由于智能系统的推理过程一般表现为一种搜索过程，因此，推理的控制策略又可分为**推理策略和搜索策略**。

➤ 推理策略

- ❖ 推理方向控制策略用于确定推理的控制方向，可分为正向推理、逆向推理、混合推理及双向推理。
- ❖ 求解策略是指仅求一个解，还是求所有解或最优解等。
- ❖ 限制策略是指对推理的深度、宽度、时间、空间等进行的限制。
- ❖ 冲突消解策略是指当推理过程有多条知识可用时，如何从这多条可用知识中选出一条最佳知识用于推理的策略。

➤ 搜索策略

主要解决推理线路、推理效果、推理效率等问题。

目 录

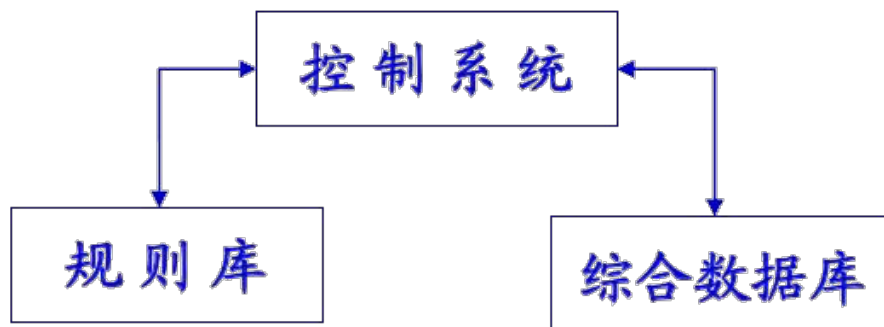
3.1 概述

3.2 产生式系统

3.3 自然演绎推理

3.4 归结演绎推理

3.2.1 产生式系统的基本结构



➤ 综合数据库DB(Data Base)

❖ 存放推理过程的各种当前信息

比如：问题的初始状态；输入的事实；
中间结论及最终结论

❖ 作为推理过程选择可用规则的依据

推理过程中某条规则是否可用，是通过该规则的前提与DB中的已知事实的匹配来确定的。可匹配的规则称为可用规则。利用可用规则进行推理，将会得到一个结论。该结论若不是目标，将作为新的事实放入DB，成为以后推理的已知事实。

➤ 规则库RB(Rule/Knowledge Base)

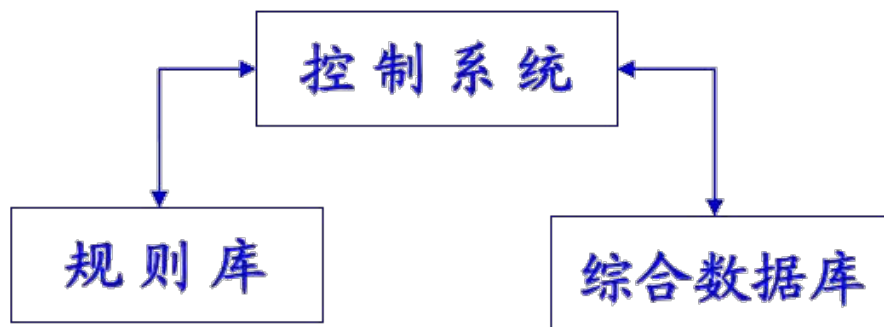
❖ 作用

用于存放推理所需要的所有规则，是整个产生式系统的知识集，是产生式系统能够进行推理的根本。

❖ 要求

知识的完整性、一致性、准确性、灵活性和可组织性

3.2.1 产生式系统的基本结构



➤ 控制系统(Control system)

❖ 控制系统的主要作用

亦称推理机，用于控制整个产生式系统的运行，决定问题求解过程的推理线路。

❖ 控制系统的主要任务

选择匹配：按一定策略从规则库中选择规则与综合数据库中的已知事实进行匹配。匹配是指把所选规则的前提与综合数据库中的已知事实进行比较，若事实库中存的事实与所选规则前提一致，则称匹配成功，该规则为可用；否则，称匹配失败，该规则不可用。

冲突消解：对匹配成功的规则，按照某种策略从中选出一条规则执行。

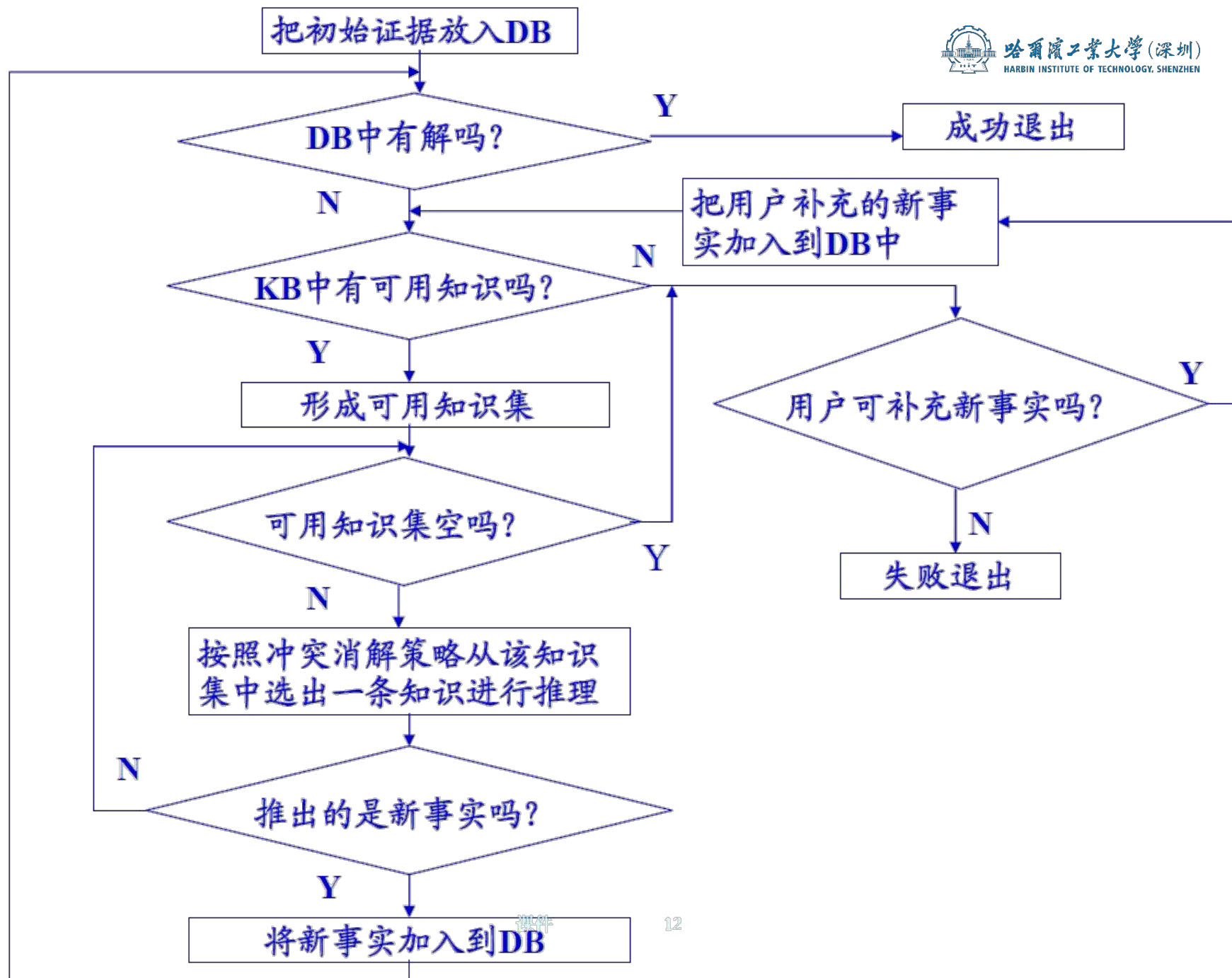
执行操作：对所执行的规则，若其后件为一个或多个结论，则把这些结论加入综合数据库；若其后件为一个或多个操作时，执行这些操作。

终止推理：检查综合数据库中是否包含有目标，若有，则停止推理。

路径解释：在问题求解过程中，记住应用过的规则序列，以便最终能够给出问题的解的路径。

3.2.2 正向推理

已知事实出发、正向使用
规则，亦称为数据驱动推
理或前向链推理



3.2.2 正向推理

□ 算法描述

- (1) 把用户提供的初始证据放入综合数据库；
- (2) 检查综合数据库中是否包含了问题的解，若已包含，则求解结束，并成功推出；否则执行下一步；
- (3) 检查知识库中是否有可用知识，若有，形成当前可用知识集，执行下一步；否则转(5)。
- (4) 按照某种冲突消解策略，从当前可用知识集中选出一条规则进行推理，并将推出的新事实加入综合数据库种，然后转(2)。
- (5) 询问用户是否可以进一步补充新的事实，若可补充，则将补充的新事实加入综合数据库中，然后转(3)；否则表示无解，失败退出。

至于如何根据综合数据库中的事实到知识库中选取可用知识，当知识库中有多条知识可用时应该先使用那一条知识等。这些问题涉及到了知识的匹配方法和冲突消解策略。

3.2.2 正向推理

□ 正向推理实例

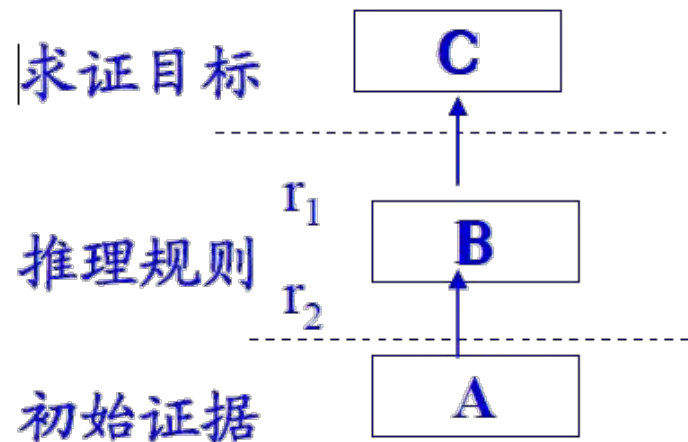
例: 请用正向推理完成以下问题的求解

假设知识库中包含有以下2条规则:

r1 : IF B THEN C

r2 : IF A THEN B

已知初始证据A, 求证目标C。



3.2.2 正向推理

□ 正向推理实例

例: 请用正向推理完成以下问题的求解

假设知识库中包含有以下2条规则:

r1 : IF B THEN C

r2 : IF A THEN B

已知初始证据A, 求证目标C。

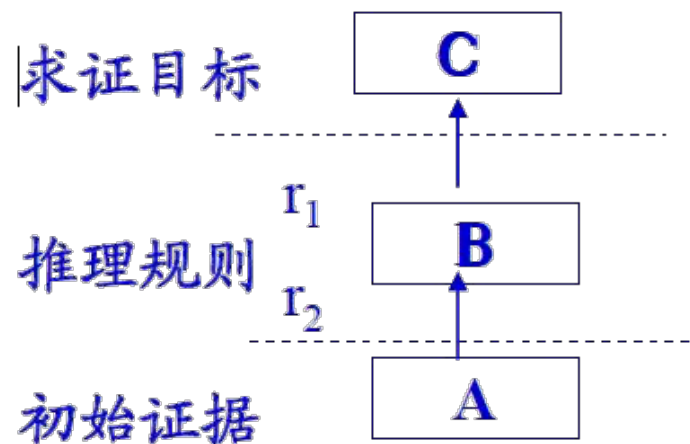
解: 推理过程如下:

推理开始前, 综合数据库为空。推理开始后, 先把A放入综合数据库, 然后检查综合数据库中是否含有该问题的解, 回答为“N”。

接着检查知识库中是否有可用知识, 显然r2可用, 形成仅含r2的知识集。从该知识集中取出r2, 推出新的实事B, 将B加入综合数据库, 检查综合数据库中是否含有目标C, 回答为“N”。

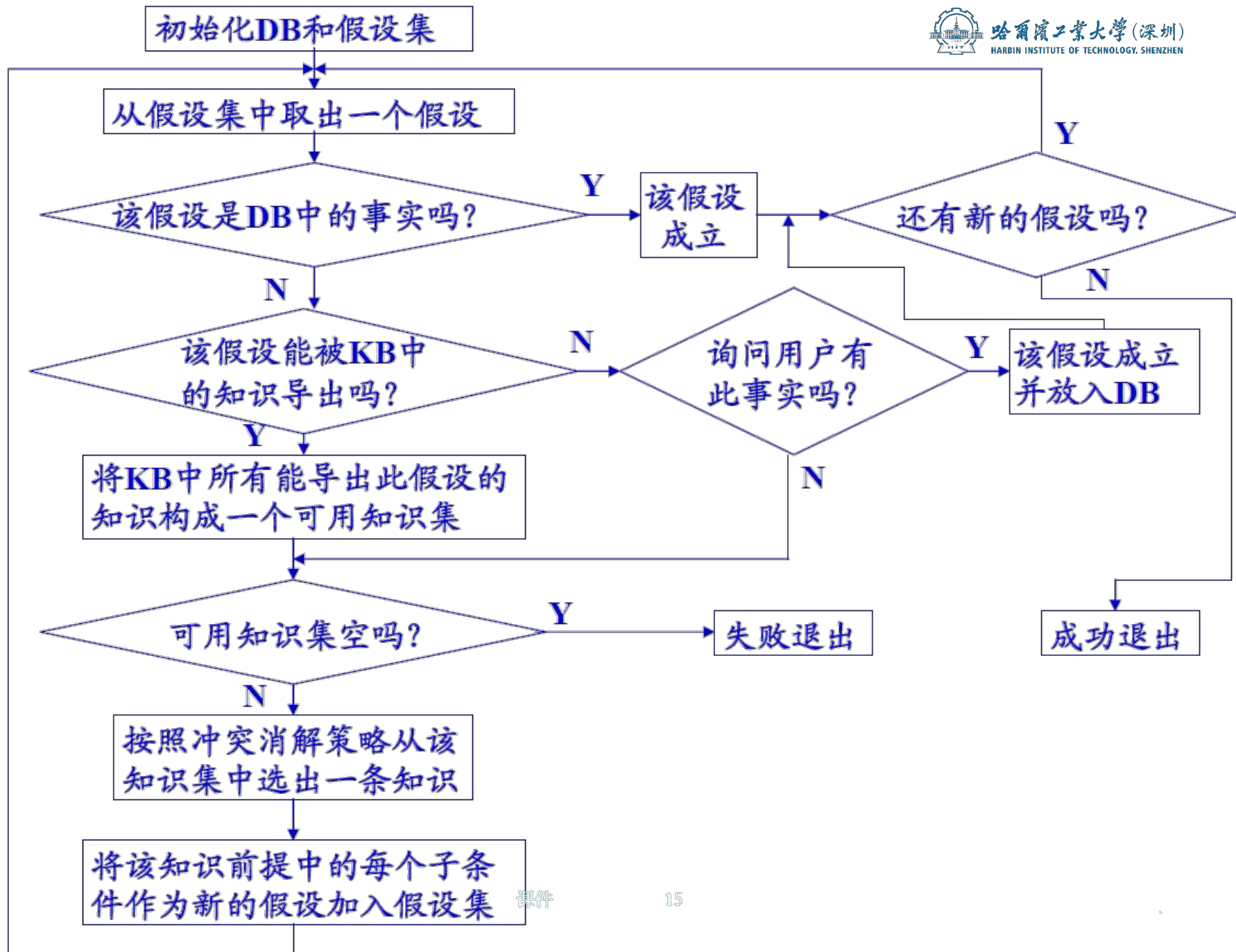
再检查知识库中是否有可用知识, 此时由于B的加入使得r1为可用, 形成仅含r1的知识集。从该知识集中取出r1, 推出新的实事C, 将C加入综合数据库, 检查综合数据库中是否含有目标C, 回答为“Y”。

它说明综合数据库中已经含有问题的解, 推理成功结束, 目标C得证。



3.2.3 逆向推理

从某个假设目标出发，
逆向使用规则，亦称为
目标驱动推理或**逆向链推理**



3.2.3 拟向推理

□ 算法描述

- (1) 将要求证的目标（称为假设）构成一个假设集；
- (2) 从假设集中选出一个假设，检查该假设是否在综合数据库中，若在，则该假设成立，此时，若假设集为空，则成功退出，否则仍执行(2)；若该假设不在数据库中，则执行下一步；
- (3) 检查该假设是否可由知识库的某个知识导出，若不能由某个知识导出，则询问用户该假设是否为可由用户证实的原始事实，若是，该假设成立，并将其放入综合数据库再重新寻找新的假设，若不是，则转(5)；若能由某个知识导出，则执行下一步；
- (4) 将知识库中可以导出该假设的所有知识构成一个可用知识集；
- (5) 检查可用知识集是否为空，若是，失败退出；否则执行下一步；
- (6) 按冲突消解策略从可用知识集中取出一个知识，继续；
- (7) 将该知识的前提中的每个子条件都作为新的假设放入假设集，然后转(2)。

3.2.3 逆向推理

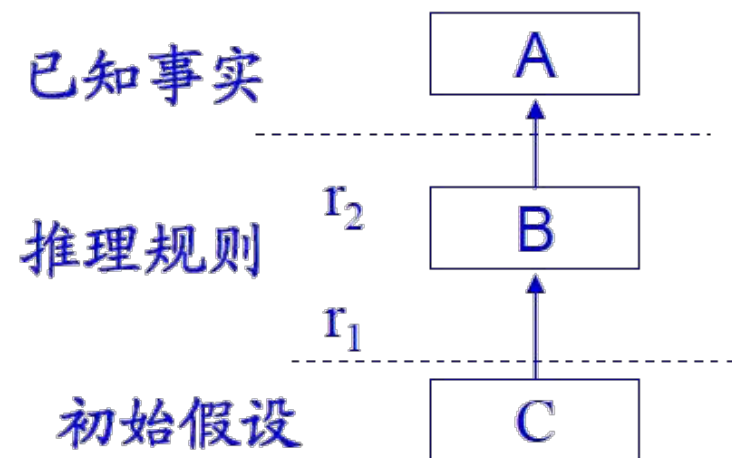
□ 正向推理实例

例: 请用逆向推理完成以下问题的求解
假设知识库中包含有以下2条规则:

r1 : IF B THEN C

r2 : IF A THEN B

已知初始证据A, 求证目标C。



3.2.3 逆向推理

□ 正向推理实例

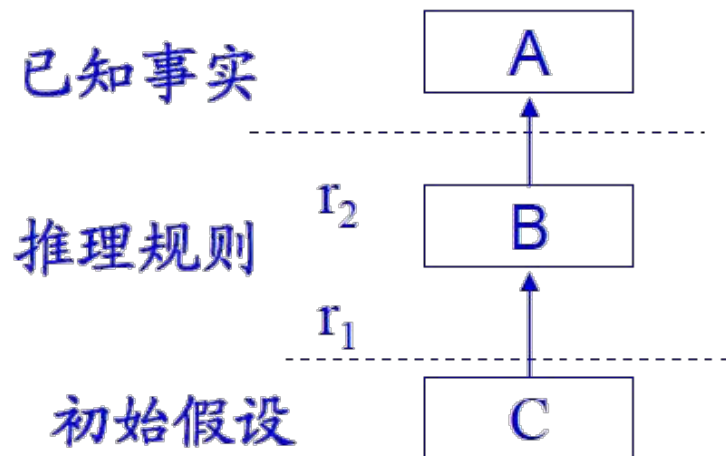
例: 请用逆向推理完成以下问题的求解

假设知识库中包含有以下2条规则:

r1 : IF B THEN C

r2 : IF A THEN B

已知初始证据A, 求证目标C。



解: 推理过程如下:

推理开始前, 综合数据库和假设集均为空。

推理开始后, 先将初始证据A和目标C分别放入综合数据库和假设集, 然后从假设集中取出一个假设C, 查找C是否为综合数据库中的已知事实, 回答为“N”。

再检查C是否能被知识库中的知识所导出, 发现C可由r1导出, 于是r1被放入可用知识集。由于知识库中只有r1可用, 故可用知识集中仅含r1。

接着从可用知识集中取出r1, 将其前提条件B作为新的假设放入假设集。从假设集中取出B, 检查B是否为综合数据库中的实事, 回答为“N”。再检查B是否能被知识库中的知识所导出, 发现B可由r2导出, 于是r2被放入可用知识集。由于知识库中只有r2可用, 故可用知识集中仅含r2。

从可用知识集中取出r2, 将其前提条件A作为新的假设放入假设集。然后从假设集中取出A, 检查A是否为综合数据库中的实事, 回答为“Y”。

这说明该假设成立, 由于无新的假设, 故推理过程成功结束, 于是目标C得证。

3.2.4 阐述式系统的示例

□ 基于规则的动物识别系统

该系统可以识别老虎、金钱豹、斑马、长颈鹿、鸵鸟、企鹅、信天翁这7种动物。其规则库包含如下15条规则：

r1 IF 动物有毛发 THEN 动物是哺乳动物

r2 IF 动物有奶 THEN 动物是哺乳动物

r3 IF 动物有羽毛 THEN 动物是鸟

r4 IF 动物会飞 AND 动物会下蛋 THEN 动物是鸟

r5 IF 动物吃肉 THEN 动物是食肉动物

r6 IF 动物有犬齿 AND 动物有爪 AND 该物眼盯前方 THEN 动物是食肉动物

r7 IF 动物是哺乳动物 AND 动物有蹄 THEN 动物是有蹄类动物

r8 IF 动物是哺乳动物 AND 动物是反刍动物 THEN 动物是有蹄类动物

r9 IF 动物是哺乳动物 AND 动物是食肉动物 AND 动物是黄褐色 AND 动物身上有暗斑点 THEN 动物是金钱豹

3.2.4 阐述式系统的示例

□ 基于规则的动物识别系统

r10 IF 动物是哺乳动物 AND 动物是食肉动物 AND 动物是黄褐色 AND 动物身上有黑色条纹 THEN 动物是虎

r11 IF 动物是有蹄类动物 AND 动物有长脖子 AND 动物有长腿 AND 动物身上有暗斑点 THEN 动物是长颈鹿

r12 IF 动物是有蹄类动物 AND 动物身上有黑色条纹 THEN 动物是斑马

r13 IF 动物是鸟 AND 动物有长脖子 AND 动物有长腿 AND 动物不会飞 AND 动物有黑白二色 THEN 动物是鸵鸟

r14 IF 动物是鸟 AND 动物会游泳 AND 动物不会飞 AND 动物有黑白二色 THEN 动物是企鹅

r15 IF 动物是鸟 AND 动物善飞 THEN 动物是信天翁

其中， r_i ($i=1,2,\dots,15$)是规则的编号

初始综合数据库包含的事实有：

动物有暗斑点，动物有长脖子，动物有长腿，动物有奶，动物有蹄

3.2.4 阐述式系统的示例

□ 基于规则的动物识别系统推理过程

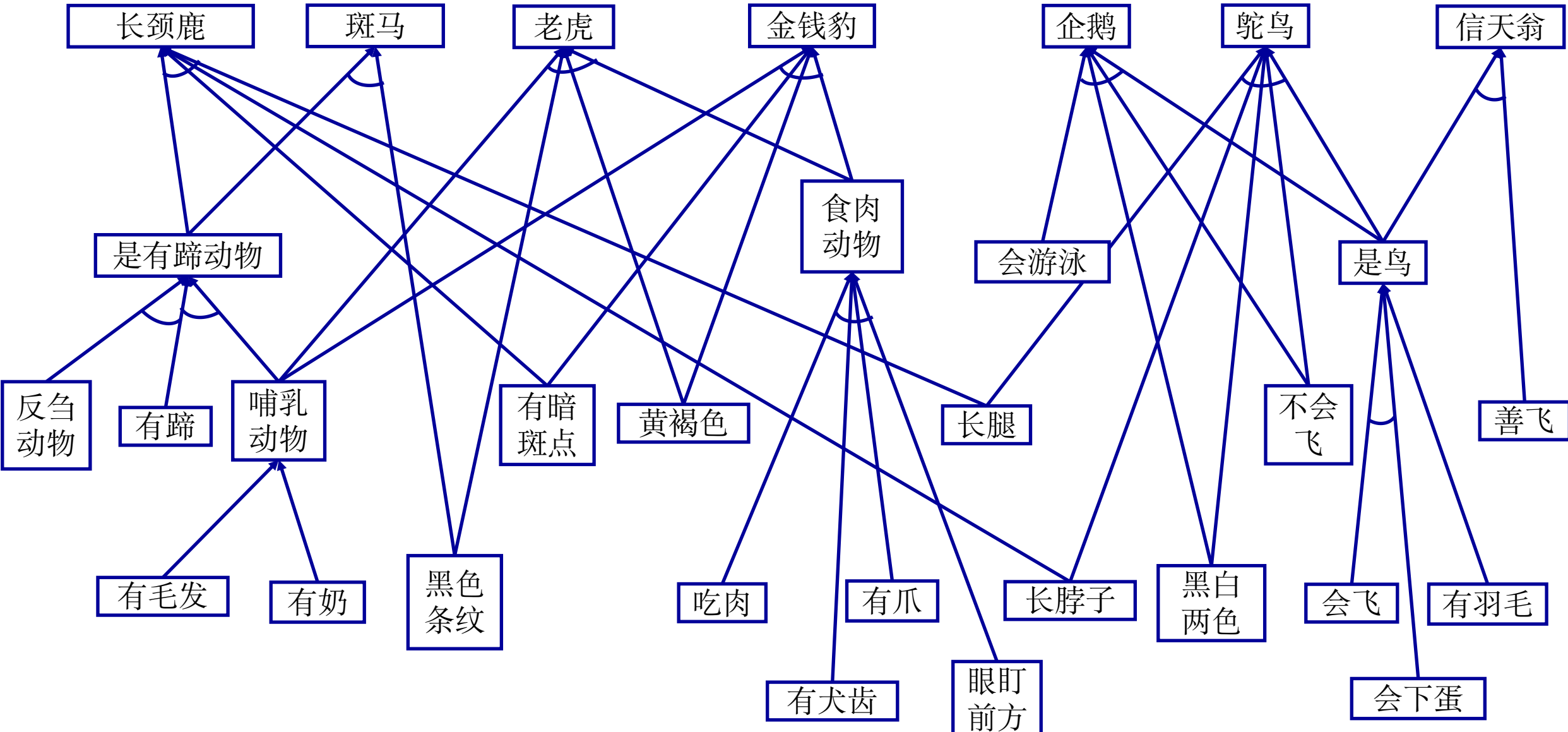
(1) 先从规则库中取出第一条规则 r_1 ，检查其前提是否可与综合数据库中的已知事实相匹配。 r_1 的前提是“动物有毛发”，但事实库中无此事实，故匹配失败。然后取 r_2 ，该前提可与已知事实“动物有奶”相匹配， r_2 被执行，并将其结论“动物是哺乳动物”作为新的事实加入到综合数据库中。此时，综合数据库的内容为：动物有暗斑，动物有长脖子，动物有长腿，动物有奶，动物有蹄，动物是哺乳动物

(2) 再从规则库中取 r_3 ， r_4 ， r_5 ， r_6 进行匹配，均失败。接着取 r_7 ，该前提与已知事实“动物是哺乳动物”相匹配， r_7 被执行，并将其结论“动物是有蹄类动物”作为新的事实加入到综合数据库中。此时，综合数据库的内容变为：动物有暗斑，动物有长脖子，动物有长腿，动物有奶，动物有蹄，动物是哺乳动物，动物是有蹄类动物

(3) 此后， r_8 ， r_9 ， r_{10} 均匹配失败。接着取 r_{11} ，该前提“动物是有蹄类动物AND 动物有长脖子AND 动物有长腿AND 动物身上有暗斑”与已知事实相匹配， r_{11} 被执行，并推出“动物是长颈鹿”。由于“长颈鹿”已是目标集合中的一个具体动物，即已推出最终结果，故问题求解过程结束。

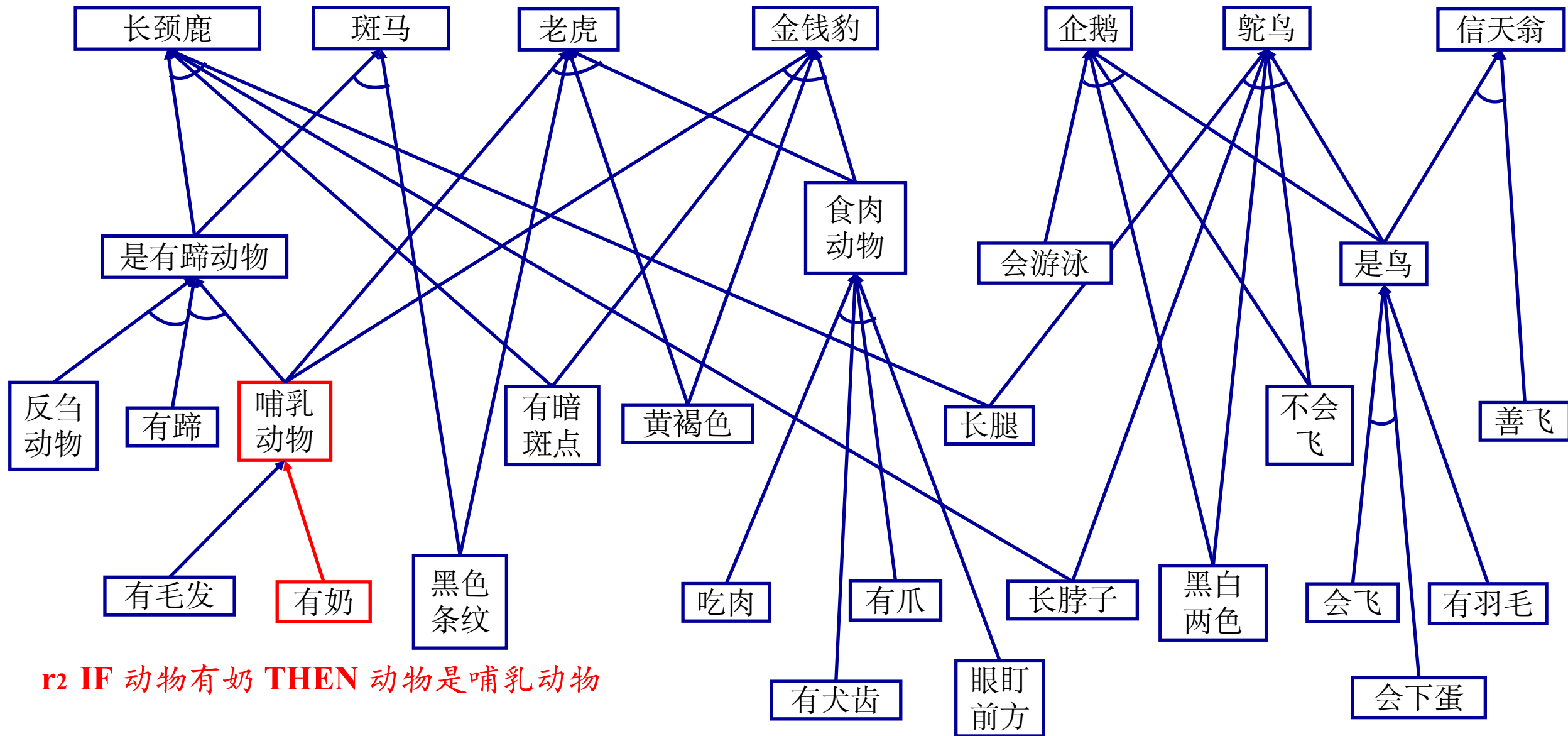
3.3.4 阐述式系统的示例

初始综合数据库包含事实: 动物有暗斑点, 动物有长脖子, 动物有长腿, 动物有奶, 动物有蹄



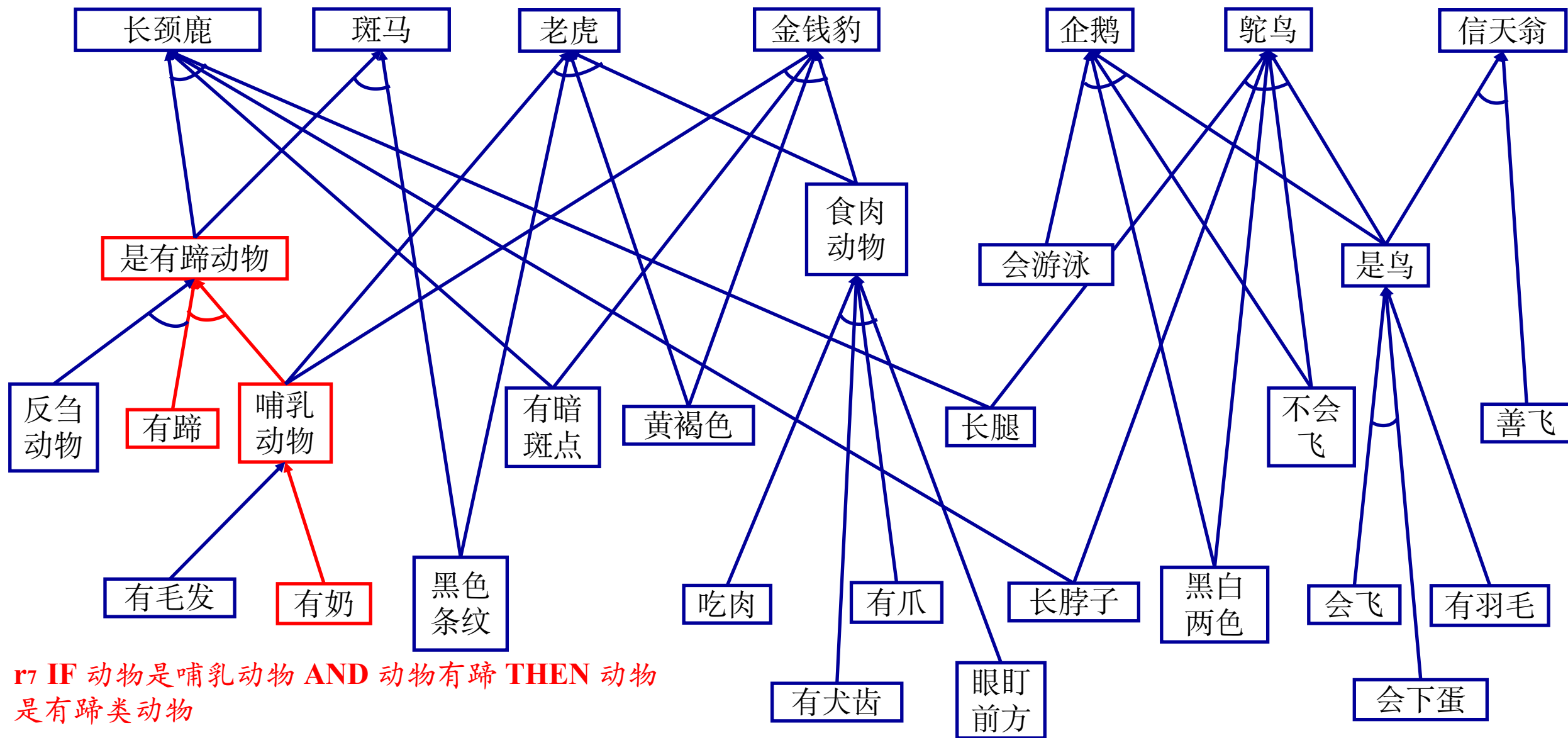
3.2.4 阐述式系统的示例

初始综合数据库包含事实: 动物有暗斑点, 动物有长脖子, 动物有长腿, 动物有奶, 动物有蹄



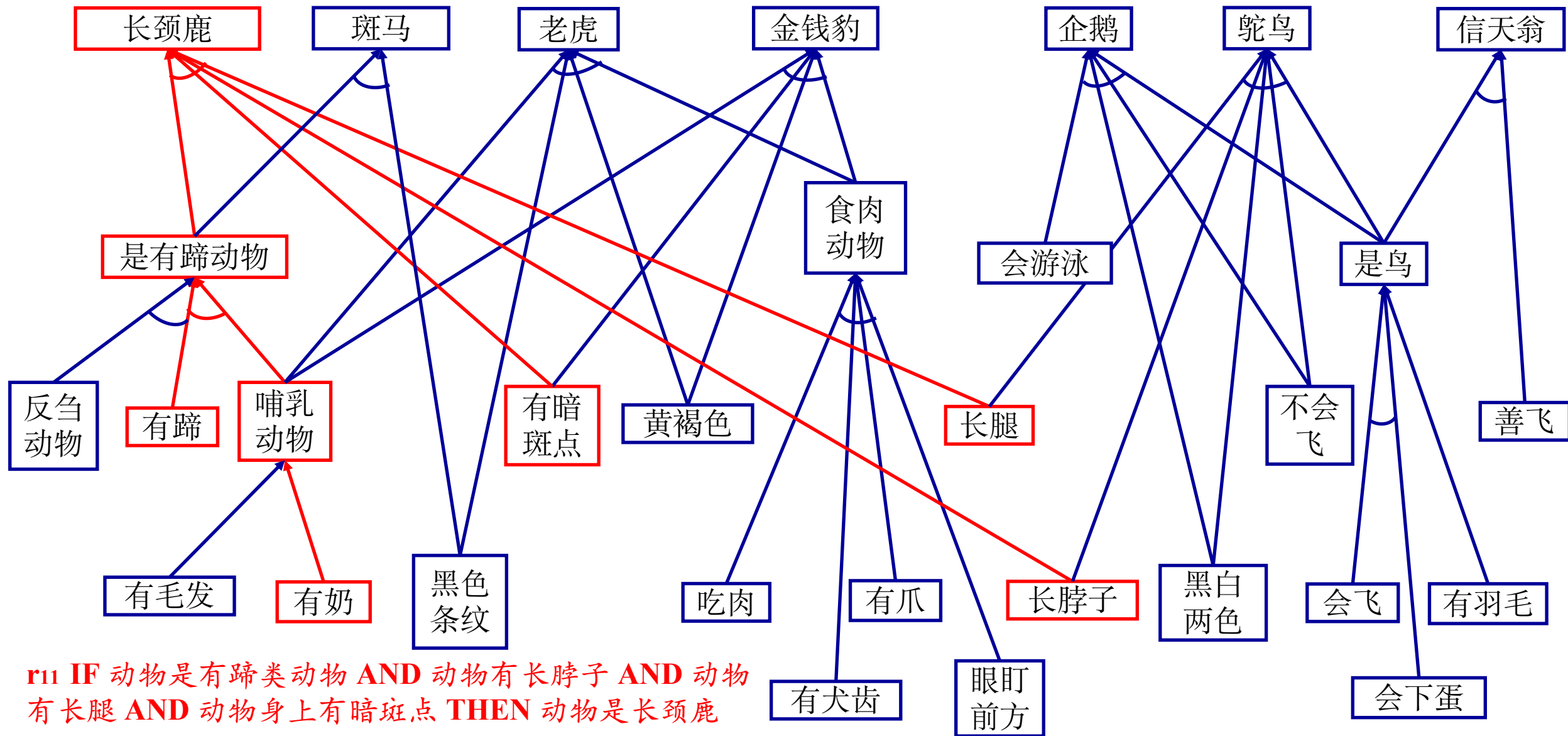
哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

r7 IF 动物是哺乳动物 AND 动物有蹄 THEN 动物是有蹄类动物



3.2.4 阐述式系统的示例

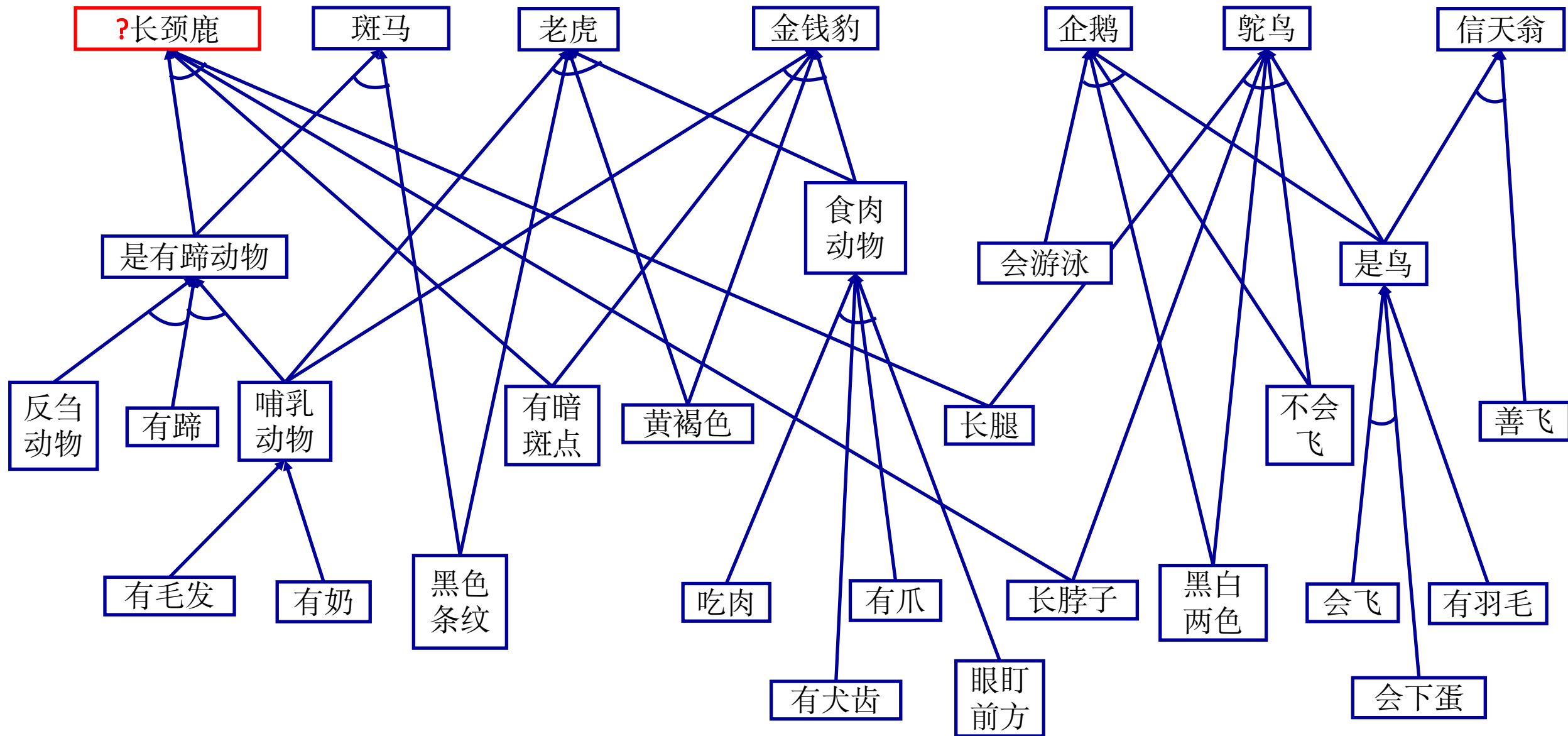
初始综合数据库包含事实: 动物有暗斑点, 动物有长脖子, 动物有长腿, 动物有奶, 动物有蹄



r11 IF 动物是有蹄类动物 AND 动物有长脖子 AND 动物有长腿 AND 动物身上有暗斑点 THEN 动物是长颈鹿

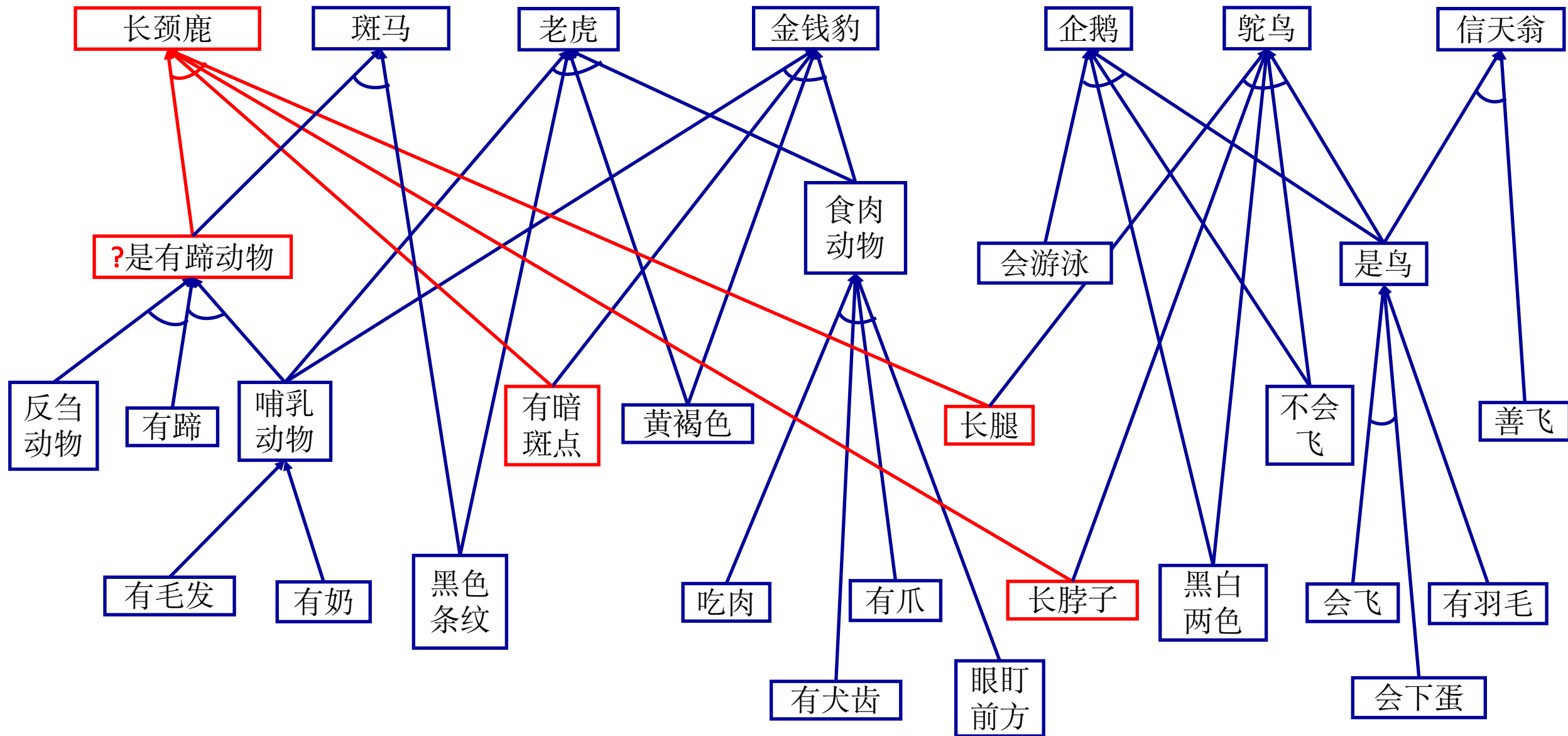
3.2.4 阐述式系统的示例

初始综合数据库包含事实: 动物有暗斑点, 动物有长脖子, 动物有长腿, 动物有奶, 动物有蹄



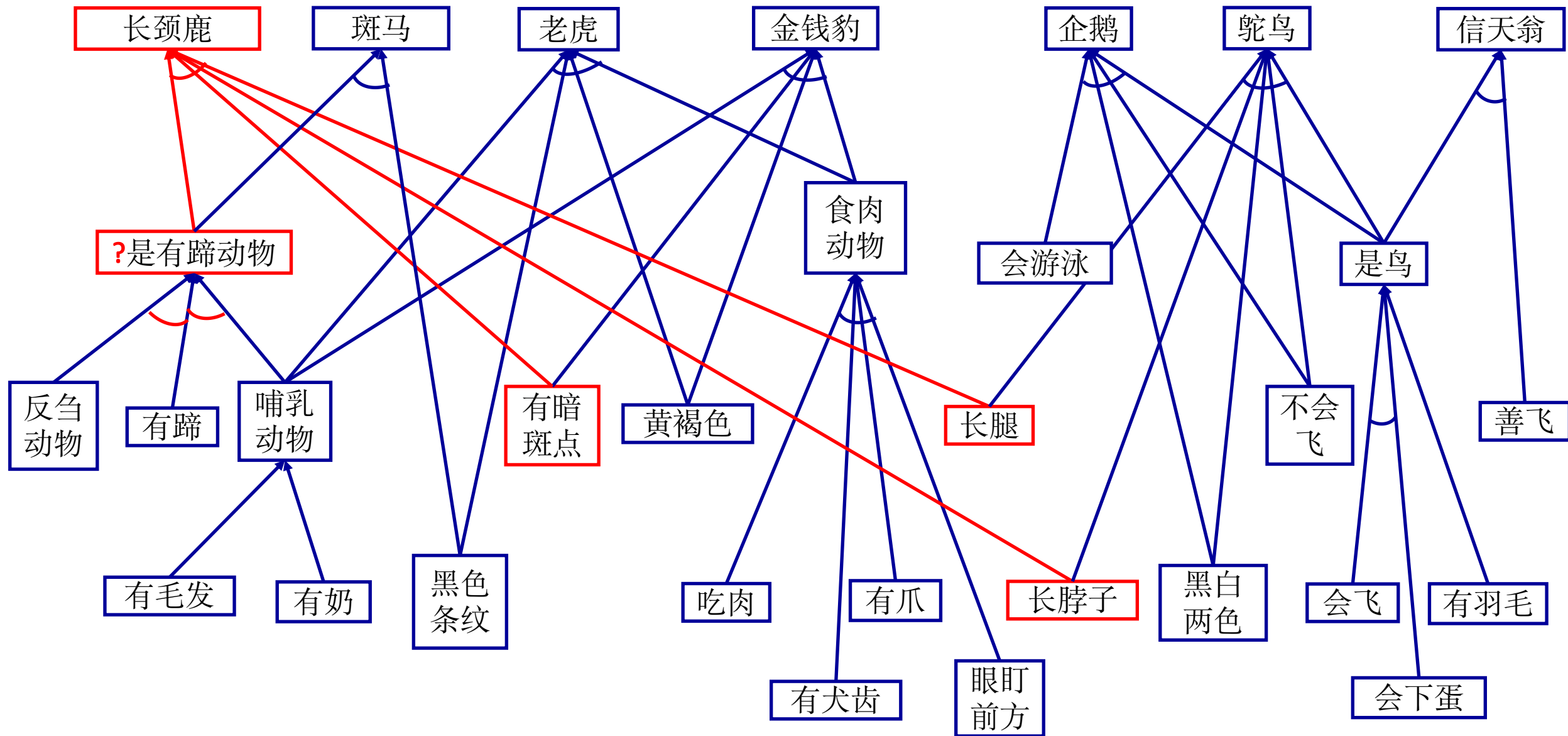
3.2.4 阐述式系统的示例

初始综合数据库包含事实: 动物有暗斑点, 动物有长脖子, 动物有长腿, 动物有奶, 动物有蹄



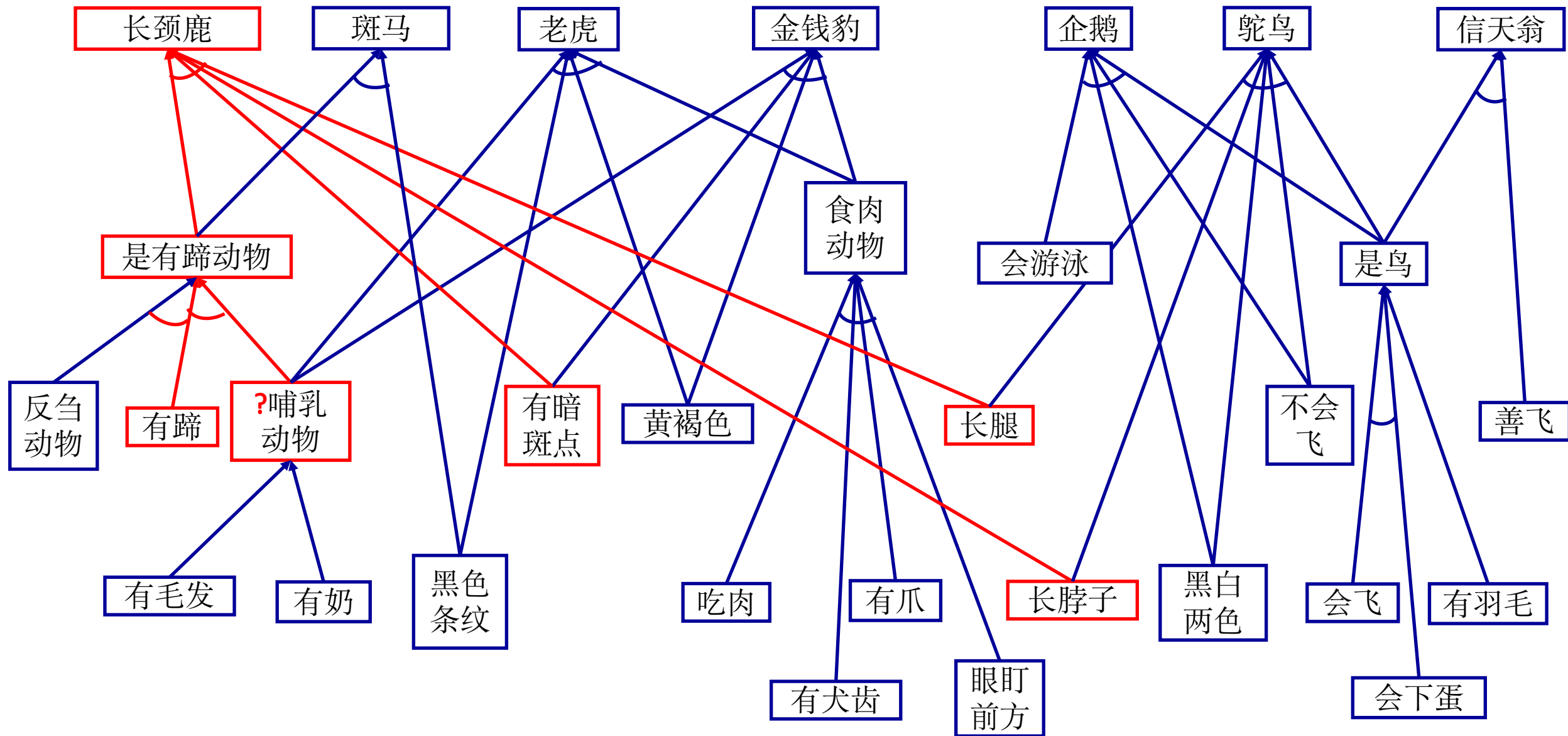
3.2.4 阐述式系统的示例

初始综合数据库包含事实: 动物有暗斑点, 动物有长脖子, 动物有长腿, 动物有奶, 动物有蹄



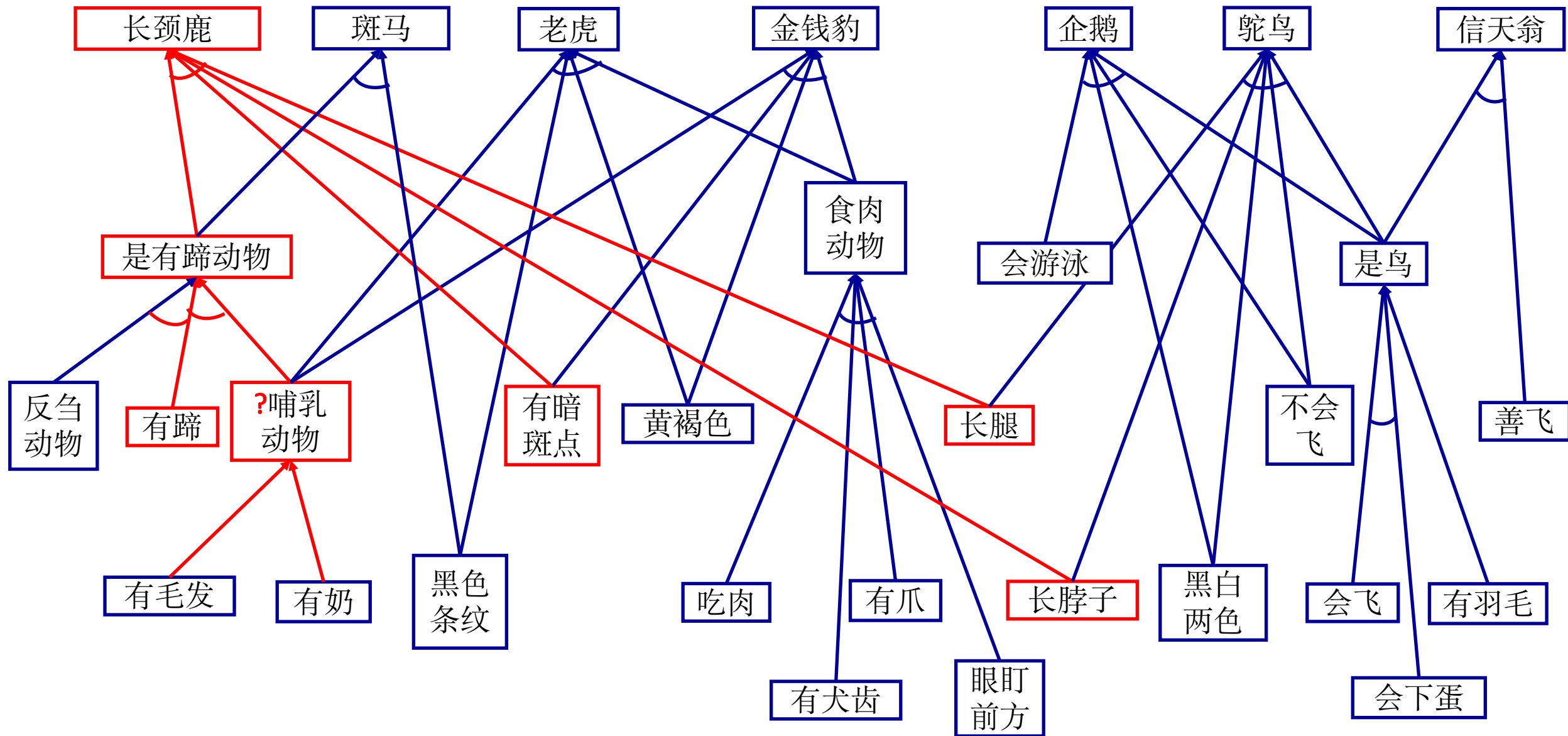
3.2.4 阐述式系统的示例

初始综合数据库包含事实: 动物有暗斑点, 动物有长脖子, 动物有长腿, 动物有奶, 动物有蹄



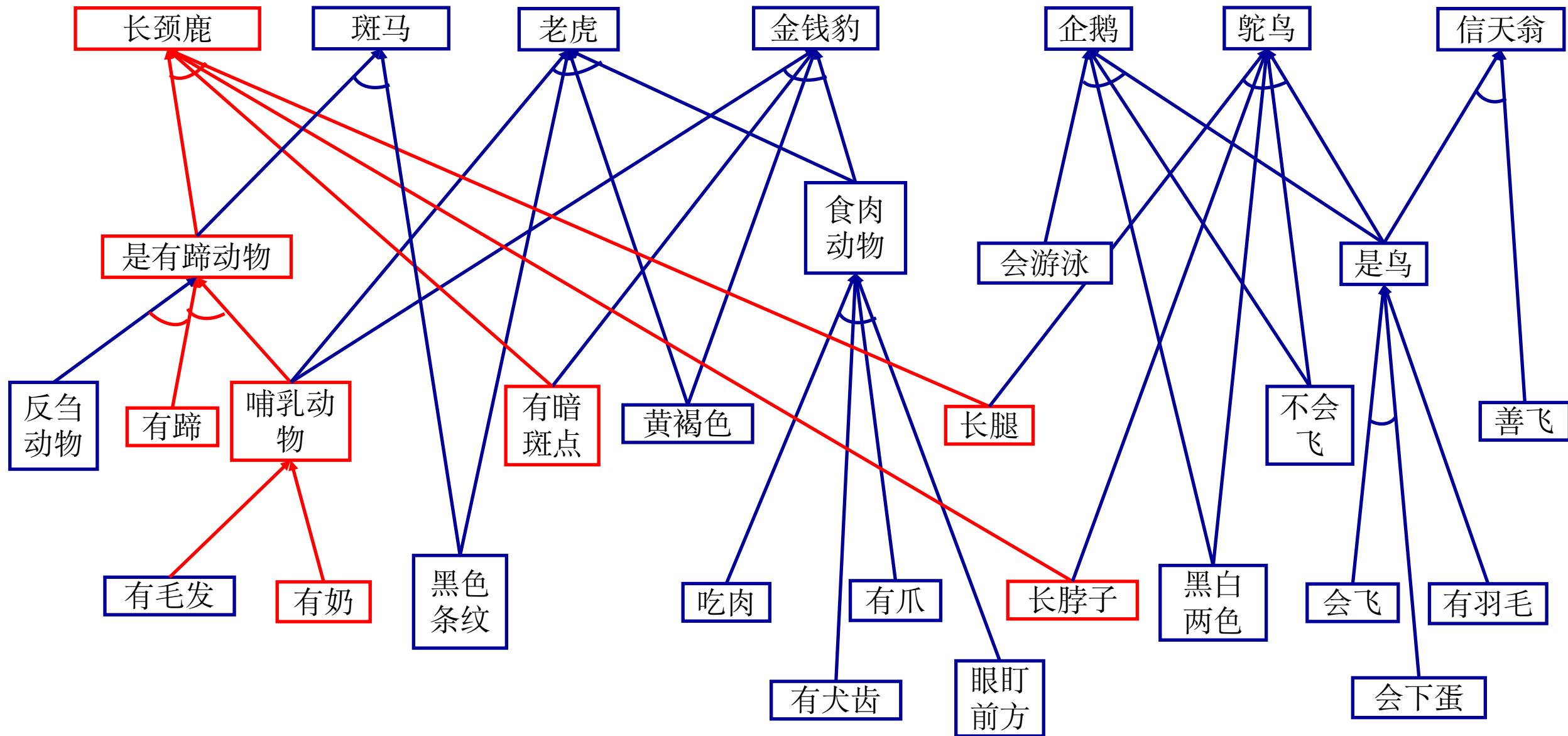
3.2.4 阐述式系统的示例

初始综合数据库包含事实: 动物有暗斑点, 动物有长脖子, 动物有长腿, 动物有奶, 动物有蹄



3.2.4 阐述式系统的示例

初始综合数据库包含事实: 动物有暗斑点, 动物有长脖子, 动物有长腿, 动物有奶, 动物有蹄



3.2.5 产生式系统小节

(1) 正向推理的特性

正向推理的主要优点是比较直观，主要缺点是推理无明确的目标，求解问题时可能会执行许多与解无关的操作，导致推理效率较低。

(2) 逆向推理的特性

逆向推理的主要优点是不必寻找和使用那些与假设目标无关的信息和规则，推理过程的目标明确，主要缺点是当用户对解的情况认识不清时，由系统自主选择假设目标的盲目性比较大，若选择不好，会影响系统效率。

(3) 双向推理方法

为互相取长补短，可以把正向和逆向结合起来使用，采用双向推理的方式。双向推理有多种不同的实现方法，可以采用先正向后逆向，也可以采用先逆向后正向，还可以采用随机选择正向和逆向的推理方法。

(4) 推理过程的不唯一性

从前面的推理算法可以看出，无论是正向推理还是逆向推理，当可用规则集中有多条规则可用时，不同的冲突消解策略将导致不同的规则使用顺序，因此其推理过程是不唯一的。

目 录

3.1 概述

3.2 产生式系统

3.3 自然演绎推理

3.4 归结演绎推理

3.3.1 一阶谓词逻辑基础

□ 定义1 谓词公式的解释

设D是谓词公式P的非空个体域，若对P中的常量，函数和谓词按如下规定赋值：

- 为每个个体**常量**指派D中的一个元素；
- 为每个n**元函数**指派一个从 D^n 到D的一个映射，其中

$$D^n = \{(x_1, x_2, \dots, x_n) | x_1, x_2, \dots, x_n \in D\}$$

- 为每个n**元谓词**指派一个从 D^n 到{F,T}的映射，则称这些指派为P在D上的一个解释

3.3.1 一阶谓词逻辑基础

例：设个体域 $D = \{1, 2\}$ ，求公式 $A = (\forall x)(\exists y)P(x, y)$ 在 D 上的一个解释，并指出在该解释下公式 A 的真值。

解：由于公式 A 没有包含个体常量和函数，因此可以直接为谓词指派真值，设有

$P(1, 1)$	$P(1, 2)$	$P(2, 1)$	$P(2, 2)$
T	F	T	F

在这个指派下，公式 A 的真值为T。

3.3.1 一阶谓词逻辑基础

例：设个体域 $D = \{1, 2\}$ ，求公式 $B = (\forall x)P(f(x), a)$ 在 D 上的一个解释，并指出在该解释下公式 B 的真值。

解：设对个体常量 a 和函数 $f(x)$ 的值指派为：

a	$f(1)$	$f(2)$
1	1	2

对谓词的真值指派为：

$P(1, 1)$	$P(1, 2)$	$P(2, 1)$	$P(2, 2)$
T	F	T	F

在该解释下公式 B 的真值是T。

3.3.1 一阶谓词逻辑基础

- **定义2 谓词公式的永真性：** 如果谓词公式 P 对非空个体域 D 上的任一解释都取得真值 T ，则称 P 在 D 上是永真的；如果 P 在任何非空个体域上都是永真的，则称 P 是永真。
- **定义3 谓词公式的可满足性：** 对于谓词公式 P ，如果至少存在 D 上的一个解释，使公式 P 在此解释下的真值为 T ，则称公式 P 在 D 上是可满足的。
- **定义4 谓词公式的永假性：** 如果谓词公式 P 对非空个体域 D 上的任一解释都取真值 F ，则称 P 在 D 上是永假的；如果 P 在任何非空个体域上均是永假的，则称 P 永假。

3.3.1 一阶谓词逻辑基础

□ **定义5 谓词公式的等价性：** 设P与Q是D上的两个谓词公式，若对D上的任意解释，P与Q都有相同的真值，则称P与Q在D上是等价的。如果D是任意非空个体域，则称P与Q是等价的，记作 $P \Leftrightarrow Q$ 。

常用的等价式有：

(1) 双重否定律 $\neg\neg P \Leftrightarrow P$

(2) 交换律 $(P \vee Q) \Leftrightarrow (Q \vee P), (P \wedge Q) \Leftrightarrow (Q \wedge P)$

(3) 结合律 $(P \vee Q) \vee R \Leftrightarrow P \vee (Q \vee R)$
 $(P \wedge Q) \wedge R \Leftrightarrow P \wedge (Q \wedge R)$

(4) 分配律 $P \vee (Q \wedge R) \Leftrightarrow (P \vee Q) \wedge (P \vee R)$
 $P \wedge (Q \vee R) \Leftrightarrow (P \wedge Q) \vee (P \wedge R)$

3.3.1 一阶谓词逻辑基础

(5) 狄摩根定律 $\neg(P \vee Q) \Leftrightarrow \neg P \wedge \neg Q, \quad \neg(P \wedge Q) \Leftrightarrow \neg P \vee \neg Q$

(6) 吸收律 $P \vee (P \wedge Q) \Leftrightarrow P, \quad P \wedge (P \vee Q) \Leftrightarrow P$

(7) 补余律 $P \vee \neg P \Leftrightarrow T, \quad P \wedge \neg P \Leftrightarrow F$

(8) 连词化归律

$$P \rightarrow Q \Leftrightarrow \neg P \vee Q$$
$$P \Leftrightarrow Q \Leftrightarrow (P \rightarrow Q) \wedge (Q \rightarrow P)$$
$$P \Leftrightarrow Q \Leftrightarrow (P \wedge Q) \vee (\neg Q \wedge \neg P)$$

(9) 量词转换律 $\neg(\exists x)P \Leftrightarrow (\forall x)(\neg P), \quad \neg(\forall x)P \Leftrightarrow (\exists x)(\neg P)$

(10) 量词分配律

$$(\forall x)(P \wedge Q) \Leftrightarrow (\forall x)P \wedge (\forall x)Q$$
$$(\exists x)(P \vee Q) \Leftrightarrow (\exists x)P \vee (\exists x)Q$$

3.3.1 一阶谓词逻辑基础

□ **定义6 永真蕴含式：**对谓词公式P和Q，如果 $P \rightarrow Q$ 永真，则称P永真蕴含Q，且称Q为P的逻辑结论，P为Q的前提，记作 $P \Rightarrow Q$ 。

常用的永真蕴含式如下：

(1) 化简式 $P \wedge Q \Rightarrow P, P \wedge Q \Rightarrow Q$

(2) 附加式 $P \Rightarrow P \vee Q, Q \Rightarrow P \vee Q$

(3) 析取三段论 $\neg P, P \vee Q \Rightarrow Q$

(4) 假言推理 $P, P \rightarrow Q \Rightarrow Q$

3.3.1 一阶谓词逻辑基础

常用的永真蕴含式如下:

(5) 拒取式 $\neg Q, P \rightarrow Q \Rightarrow \neg P$

(6) 假言三段论 $P \rightarrow Q, Q \rightarrow R \Rightarrow P \rightarrow R$

(7) 二难推理 $P \vee Q, P \rightarrow R, Q \rightarrow R \Rightarrow R$

(8) 全称固化 $(\forall x)P(x) \Rightarrow P(y)$

其中, y 是个体域中的任一个体, 依此可消去谓词公式中的全称量词。

(9) 存在固化 $(\exists x)P(x) \Rightarrow P(y)$

其中, y 是个体域中某一个可以使 $P(y)$ 为真的个体, 依此可消去谓词公式中的存在量词。

3.3.2 置换合一

□ 合一

- ❖ 在不同谓词公式中，往往会出现谓词名相同但其个体不同的情况，此时推理过程是不能直接进行匹配的，需要先进行置换。

$$W(a) \text{ 和 } W(x) \rightarrow Q(x)$$

对谓词 $W(a)$ 与 $W(x)$ 谓词名相同，但个体不同，不能直接进行推理。

- ❖ 要使用假言推理，首先需要找到项 a 对变元 x 的置换，使 $W(a)$ 与 $W(x)$ 一致。

这种寻找项对变元的置换，使谓词一致的过程叫做合一的过程。

3.3.2 置换合一

□ **置换**：可简单的理解为是在一个谓词公式中用置换项去替换变量。

定义7 置换是形如

$$\{t_1/x_1, t_2/x_2, \dots, t_n/x_n\}$$

的有限集合。其中， $t_1, t_2, \dots, t_n$ 是项； $x_1, x_2, \dots, x_n$ 是互不相同的变元； t_i/x_i 表示用 t_i 替换 x_i 。并且**要求 t_i 与 x_i 不能相同**， **x_i 不能循环地出现在另一个 t_i 中**。

例如， $\{a/x, c/y, f(b)/z\}$ 是一个置换。但 $\{g(z)/x, f(x)/z\}$ 不是一个置换。

原因是它在 x 与 z 之间出现了循环置换现象。即当用 $g(z)$ 置换 x ，用 $f(x)$ 置换 z 时，既没有消去 x ，也没有消去 z 。

若改为 $\{g(a)/x, f(x)/z\}$ 即可，原因是用 $g(a)$ 置换 x ，用 $f(x)$ 置换 z ，若再用一次置换，用 $g(a)$ 置换 x ，最终原 x 和 z 被 $g(a)$ 和 $f(g(a))$ 置换，则经过有限次置换消去了 x 和 z 。

通常，置换是用希腊字母 θ 、 σ 、 α 、 λ 等来表示的。

3.3.2 置换合一

□ **置换**：可简单的理解为是在一个谓词公式中用置换项去替换变量。

定义8 设 $\theta=\{t_1/x_1, t_2/x_2, \dots, t_n/x_n\}$ 是一个置换，**F**是一个谓词公式，把公式**F**中出现的所有 x_i 换成 t_i ($i=1, 2, \dots, n$)，得到一个新的公式**G**，称**G**为**F**在置换 θ 下的**例示**，记作 $G=F_\theta$ 。

3.3.2 置换合一

□ 置换的合成

设 $\theta = \{t_1/x_1, t_2/x_2, \dots, t_n/x_n\}$

$\lambda = \{u_1/y_1, u_2/y_2, \dots, u_n/y_n\}$

是两个置换，则将集合

$$\underbrace{\{t_1\lambda/x_1, t_2\lambda/x_2, \dots, t_n\lambda/x_n\}}_{\text{red line}} \underbrace{\{u_1/y_1, u_2/y_2, \dots, u_n/y_n\}}_{\text{blue line}}$$

中符合下列条件的元素删除：

(1) $t_i\lambda/x_i$ 当 $t_i\lambda=x_i$

(2) u_i/y_i 当 $y_i \in \{x_1, x_2, \dots, x_n\}$

如此得到的集合仍然是一个置换，该置换称为 θ 与 λ 的合成，记作 $\theta \cdot \lambda$ 。

3.3.2 置换合一

□ 置换合成实例

例： 设 $\theta = \{f(y) / x, z / y\}$, $\lambda = \{a / x, b / y, y / z\}$, 求 θ 与 λ 的合成。

3.3.2 置换合一

□ 置换合成实例

例： 设 $\theta = \{f(y)/x, z/y\}$, $\lambda = \{a/x, b/y, y/z\}$, 求 θ 与 λ 的合成。

解： 先求出集合

$$\begin{aligned} & \{f(y)\lambda / x, z\lambda / y, a / x, b / y, y / z\} \\ &= \{f(b) / x, y / y, a / x, b / y, y / z\} \end{aligned}$$

$y / y, a / x, b / y$, 满足定义中的条件需删除, 得:

$$\theta \cdot \lambda = \{f(b) / x, y / z\}$$

①当 $t_i\lambda = x_i$ 时, 删去 $t_i\lambda / x_i$ ($i = 1, 2, \dots, n$);

②当 $y_i \in \{x_1, x_2, \dots, x_n\}$ 时, 删去 y_i / y_i ($i = 1, 2, \dots, m$)

3.3.2 置换合一

□ **合一**：可理解为是寻找相对应变量的置换，使两个或多个谓词公式一致。

定义9 设有公式集 $F = \{F_1, F_2, \dots, F_n\}$ ，若存在一个置换 θ ，可使

$$F_1\theta = F_2\theta = \dots = F_n\theta,$$

则称 θ 是 F 的一个合一。称 $F_1, F_2, \dots, F_n$ 是可合一的。

3.3.2 置换合一

□ **合一**：可理解为是寻找相对应变量的置换，使两个或多个谓词公式一致。

定义9 设有公式集 $F=\{F_1, F_2, \dots, F_n\}$ ，若存在一个置换 θ ，可使

$$F_1\theta=F_2\theta=\dots=F_n\theta,$$

则称 θ 是 F 的一个合一。称 $F_1, F_2, \dots, F_n$ 是可合一的。

例如，设有公式集 $F=\{P(x, y, f(y)), P(a, g(x), z)\}$ ，则

$$\lambda=\{a/x, g(a)/y, f(g(a))/z\}$$

是它的一个合一。

3.3.2 置换合一

□ **合一**：可理解为是寻找相对应变量的置换，使两个或多个谓词公式一致。

定义9 设有公式集 $F=\{F_1, F_2, \dots, F_n\}$ ，若存在一个置换 θ ，可使

$$F_1\theta=F_2\theta=\dots=F_n\theta,$$

则称 θ 是 F 的一个合一。称 $F_1, F_2, \dots, F_n$ 是可合一的。

例如，设有公式集 $F=\{P(x, y, f(y)), P(a, g(x), z)\}$ ，则

$$\lambda=\{a/x, g(a)/y, f(g(a))/z\}$$

是它的一个合一。

一般情况下，一个公式集的合一是不唯一的

3.3.2 置换合一

□ 最一般合一实例

例： 设 $S = \{P(x, y, f(y)), P(a, g(x), z)\}$

S有一个最一般合一置换

$$\sigma = \{a / x, g(a) / y, f(g(a)) / z\}$$

3.3.2 置换合一

□ 最一般合一实例

例：设 $S = \{P(x, y, f(y)), P(a, g(x), z)\}$

S有一个最一般合一置换

$$\sigma = \{a / x, g(a) / y, f(g(a)) / z\}$$

对于S的任一合一置换，例如：

$$\theta = \{u / x, g(u) / y, f(g(u)) / z, u / a\}$$

存在一个替换

$$\lambda = \{u / a\}$$

使得

$$\theta = \sigma \cdot \lambda$$

3.3.2 置换合一

□ 最一般合一置换的求取算法

➤ 差异集

- ❖ 设 $F = \{F_1, F_2, \dots, F_n\}$ 是一个非空有限的公式集，从 F 中每个公式的第一个符号同时向右比较，直到发现第一个不相同的符号为止，从 F 的各个公式中取出那些以第一个不一致符号开始的最大子表达式，并以这些子表达式为元素组成一个集合 D ，称 D 为 F 的一个差异集，也称分歧集合。
- ❖ 在对两个谓词公式中的项从左到右进行比较时，那些第一个不相同的项所构成的集合，称为差异集。设 $S = \{P(x, y, z), P(x, f(a), h(b))\}$ ，则不难看出， S 有差异集 $D = \{y, f(a)\}$

3.3.2 置换合一

设 S 为非空有限公式集合，求 S 的最一般合一的算法如下：

① 置 $k=0$, $S_k=S$, $\sigma_k=\varepsilon$;

② 若 S_k 只含有一个谓词公式，则算法停止， σ_k 就是要求的最一般合一置换；

③ 求 S_k 的差异集 D_k ;

④ 若 D_k 中存在元素 x_k 和 t_k ，其中 x_k 是变元， t_k 是项且 x_k 不在 t_k 中出现，则置

$$S_{k+1} = S_k \setminus \{t_k/x_k\},$$

$$\sigma_{k+1} = \sigma_k \cdot \{t_k/x_k\}, \quad k=k+1, \text{ 然后转②};$$

⑤ 算法停止， S 的最一般合一不存在。

3.3.2 置换合一

实例1：求公式集 $S = \{P(a, x, f(g(y))), P(z, h(z, u), f(u))\}$ 的最一般合一及对应的置换。

解： $k=0$ ：

$S_0=S$, $\sigma_0=\varepsilon$, S_0 不是单元素集，求得 $D_0=\{a, z\}$ ，其中 z 是变元，且不在 a 中出现，

所以有

$$\sigma_1 = \sigma_0 \cdot \{a/z\} = \varepsilon \cdot \{a/z\} = \{a/z\}$$

$$S_1 = S_0 \{a/z\} = \{P(a, x, f(g(y))), P(a, h(a, u), f(u))\}$$

3.3.2 置换合一

实例1：求公式集 $S = \{P(a, x, f(g(y))), P(z, h(z, u), f(u))\}$ 的最一般合一及对应的置换。

解： $k=0$ ：

$S_0 = S$, $\sigma_0 = \varepsilon$, S_0 不是单元素集，求得 $D_0 = \{a, z\}$ ，其中 z 是变元，且不在 a 中出现，
所以有

$$\sigma_1 = \sigma_0 \cdot \{a/z\} = \varepsilon \cdot \{a/z\} = \{a/z\}$$

$$S_1 = S_0 \{a/z\} = \{P(a, x, f(g(y))), P(a, h(a, u), f(u))\}$$

$k=1$ ：

S_1 不是单元素集，求得 $D_1 = \{x, h(a, u)\}$ ，

$$\sigma_2 = \sigma_1 \cdot \{h(a, u)/x\} = \{a/z\} \cdot \{h(a, u)/x\} = \{a/z, h(a, u)/x\}$$

$$S_2 = S_1 \{h(a, u)/x\} = \{P(a, h(a, u), f(g(y))), P(a, h(a, u), f(u))\}$$

3.3.2 置换合一

$k=2$:

S_2 不是单元素集, $D_2 = \{g(y), u\}$,

$\sigma_3 = \sigma_2 \cdot \{g(y)/u\} = \{a/z, h(a, g(y))/x, g(y)/u\}$

$S_3 = S_2\{g(y)/u\} = \{P(a, h(a, g(y), f(g(y))), P(a, h(a, g(y)), f(g(y)))\}$
 $= \{P(a, h(a, g(y)), f(g(y)))\}$

3.3.2 置换合一

$k=2$:

S_2 不是单元素集, $D_2 = \{g(y), u\}$,

$$\sigma_3 = \sigma_2 \cdot \{g(y)/u\} = \{a/z, h(a, g(y))/x, g(y)/u\}$$

$$\begin{aligned} S_3 = S_2 \{g(y)/u\} &= \{P(a, h(a, g(y), f(g(y))), P(a, h(a, g(y)), f(g(y)))\} \\ &= \{P(a, h(a, g(y)), f(g(y)))\} \end{aligned}$$

$k=3$:

S_3 已是单元素集, 所以 σ_3 就是S的最一般合一置换。

3.3.2 置换合一

实例2: 判定 $S = \{P(x, x), P(y, f(y))\}$ 是否可合一?

3.3.2 置换合一

实例2: 判定 $S = \{P(x, x), P(y, f(y))\}$ 是否可合一?

解:

$k=0$:

$$S_0 = S, \sigma_0 = \varepsilon,$$

S_0 不是单元素集, $D_0 = \{x, y\}$

$$\sigma_1 = \sigma_0 \cdot \{y/x\} = \{y/x\}$$

$$S_1 = S_0 \{y/x\} = \{P(y, y), P(y, f(y))\}$$

3.3.2 置换合一

实例2: 判定 $S = \{P(x, x), P(y, f(y))\}$ 是否可合一?

解:

$k=0$:

$$S_0 = S, \sigma_0 = \varepsilon,$$

S_0 不是单元素集, $D_0 = \{x, y\}$

$$\sigma_1 = \sigma_0 \cdot \{y/x\} = \{y/x\}$$

$$S_1 = S_0 \{y/x\} = \{P(y, y), P(y, f(y))\}$$

$k=1$:

S_1 不是单元素集, $D_1 = \{y, f(y)\}$, 由于变元 y 在项 $f(y)$ 中出现, 所以算法停止,

S 不存在最一般合一。

3.3.3 自然演绎推理方法

□ 判断以下推理是否正确

➤ 已知：

- (1) 如果行星系统是以太阳为中心的，则金星会显示出相位变化。
- (2) 金星显示出相位变化。

推出： (3) 行星系统是以太阳为中心的。

➤ 已知：

- (1) 如果一个人看报纸就能知道新闻。
- (2) 小王没有看报纸。

推出： (3) 小王不知道新闻。

3.3.3 自然演绎推理方法

□ 演绎推理实例

从一组已知为真的事实出发，直接运用命题逻辑或谓词逻辑中的推理规则推出结论的过程。

最基本的推理规则有：

◆ 假言三段论

◆ T规则

◆ 假言推理

◆ P规则

◆ 拒取式

3.3.3 自然演绎推理方法

□ 演绎推理实例

如果一个人大学毕业，则他就具有独立生活的能力。

如果一个人具有独立生活的能力，则他就可以离开父母。

假言三段论



$$P \rightarrow Q, Q \rightarrow R \Rightarrow P \rightarrow R$$

如果一个人大学毕业，则他就可以离开父母。

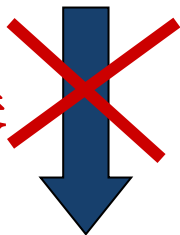
3.3.3 自然演绎推理方法

□ 演绎推理实例

如果S是音乐系学生，则S至少会演奏一样乐器。

张艺会演奏电子琴。

肯定后件错误



$$P \rightarrow Q, Q \Rightarrow P$$

张艺是音乐系学生。

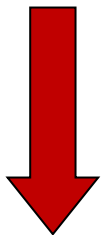
3.3.3 自然演绎推理方法

□ 演绎推理实例

如果S是音乐系学生，则S至少会演奏一样乐器。

张艺会演奏电子琴。

假言推理



$P, P \rightarrow Q$ 为真 $\Rightarrow Q$ 为真

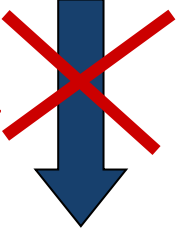
张艺至少会演奏一样乐器。

3.3.3 自然演绎推理方法

□ 演绎推理实例

如果S是音乐系学生，则S至少会演奏一样乐器。

张艺不是音乐系学生

否定前件错误  $P \rightarrow Q, \neg P \Rightarrow \neg Q$

张艺不会演奏任何一样乐器。

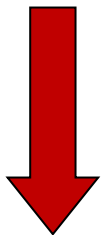
3.3.3 自然演绎推理方法

□ 演绎推理实例

如果S是音乐系学生，则S至少会演奏一样乐器。

张艺不会演奏任何乐器。

拒取式推理



$$P \rightarrow Q, \neg Q \Rightarrow \neg P$$

张艺不是音乐系学生。

3.3.3 自然演绎推理方法

□ 演绎推理实例

设已知如下事实：

$$R, S, R \rightarrow T, S \wedge T \rightarrow P, P \rightarrow Q$$

求证：Q为真。

证明：因为

$$R, R \rightarrow T \Rightarrow T$$

$$S, T \Rightarrow S \wedge T$$

$$S \wedge T, S \wedge T \rightarrow P \Rightarrow P$$

$$P, P \rightarrow Q \Rightarrow Q$$

所以Q为真。

P规则及假言推理

引入合取词

T规则及假言推理

T规则及假言推理

3.3.3 自然演绎推理方法

设有如下两个谓词公式： $W(a)$ 和 $(\forall x)(W(x) \rightarrow Q(x))$ 为真，求证 $Q(a)$ 为真

证明：由于 $W(a)$ 和 $W(x)$ 这两个谓词的个体不同，因此不能直接进行推理，需要采用置换，使它们合一。其推理过程如下：

首先对 $(\forall x)(W(x) \rightarrow Q(x))$ 进行全称固化推理，得出

$$W(y) \rightarrow Q(y)$$

然后用置换 $\theta = \{a/y\}$ 分别作用于 $W(a)$ 和 $W(y) \rightarrow Q(y)$ ，得出

$$W(a) \text{ 和 } W(a) \rightarrow Q(a)$$

最后再利用假言推理得到

$$W(a), W(a) \rightarrow Q(a) \Rightarrow Q(a)$$

即 $Q(a)$ 为真。

3.3.3 自然演绎推理方法

设已知如下事实：

(1) 如果是需要编程序的课，
王程都喜欢。

(2) 所有的程序设计语言课都
是需要编程序的课。

(3) C是一门程序设计语言课。

求证：王程喜欢C这门课。

证明：首先定义谓词

$N(x)$ x 是需要编程序的课。

$L(x, y)$ x 喜欢 y 。

$P(x)$ x 是一门程序设计语言课

把已知事实及待求解问题用谓词公式表示如下：

$N(x) \rightarrow L(\text{Wangcheng}, x)$

$(\forall x)(P(x) \rightarrow N(x))$

$P(C)$

应用推理规则进行推理：

$P(y) \rightarrow N(y)$ 全称固化

$P(C), P(y) \rightarrow N(y) \Rightarrow N(C)$ 假言推理 $\{C/y\}$

$N(C), N(x) \rightarrow L(\text{Wangcheng}, x) \Rightarrow L(\text{Wangcheng}, C)$

假言推理 $\{C/x\}$

因此，王程喜欢C这门课。

目 录

3.1 概述

3.2 产生式系统

3.3 自然演绎推理

3.4 归结演绎推理

3.4.1 归结推理方法

□ **归结演绎推理**是一种基于鲁宾逊 (Robinson) 归结原理的机器推理技术。鲁宾逊归结原理亦称为消解原理, 是鲁宾逊于1965年在海伯伦 (Herbrand) 理论的基础上提出的一种基于逻辑的“**反证法**”。

➤ **定理证明的实质**, 就是要对前提P和结论Q, 证明 $P \rightarrow Q$ 永真。

要证明 $P \rightarrow Q$ 永真, 就是要证明 $P \rightarrow Q$ 在任何一个非空的个体域上都是永真的。这将是非常困难的, 甚至是不可实现的。

➤ 为此, 人们进行了大量的探索, 后来发现可以**采用反证法的思想**, 把关于**永真性的证明转化为关于不可满足性的证明**。

即要证明 $P \rightarrow Q$ 永真, 只要能够证明 $P \wedge \neg Q$ 是不可满足的就可以了(原因是一

$$(P \rightarrow Q) \Leftrightarrow \neg(\neg P \vee Q) \Leftrightarrow P \wedge \neg Q)$$

这方面最有成效的工作就是鲁宾逊归结原理。它使定理证明的机械化成为现实。

3.4.1 归结推理方法

□ 范式是谓词公式的标准形式。

前束范式： 设F是一个谓词公式，如果其中的所有量词均**非否定**出现在公式的**最前面**，而它们的辖域为整个公式，则称F为前束范式。

前束范式一般可写成

$$(Q_1x_1)(Q_2x_2) \cdots (Q_nx_n)M(x_1, x_2, \cdots, x_n)$$

其中 $Q_i \in \{\forall, \exists\}$, $(i = 1, 2, \cdots, n)$, $M(x_1, x_2, \cdots, x_n)$ 中不含有任何量词。

例如, $(\forall x)(\forall y)(\exists z)(P(x) \wedge Q(y, z) \vee R(x, z))$ 是前束范式。

任一谓词公式均可化为与其对应的前束范式，其化简方法将在后面子句集的化简中讨论。

3.4.1 归结推理方法

➤ **Skolem标准型**：从前束型范式中消去全部存在量词所得到的公式，并将谓词公式化为合取范式，这时所得的式子称为原公式的Skolem标准型。

➤ **Skolem标准型的一般形式为**：

$$(\forall x_1)(\forall x_2) \cdots (\forall x_n)M(x_1, x_2, \dots, x_n)$$

其中 $M(x_1, x_2, \dots, x_n)$ 中不含有任何量词，且为合取范式。

3.4.2 子句集及其应用

□ 子句与子句集

归结演绎推理是在子句集的基础上讨论问题的。因此，讨论归结演绎推理之前，需要先讨论子句集的有关概念。

➤ 子句和子句集

◆ **定义10** 原子谓词公式及其否定统称为**文字**。

例如， $P(x)$ 、 $Q(x)$ 、 $\neg P(x)$ 、 $\neg Q(x)$ 等都是文字。

◆ **定义11** 任何文字的**析取式**称为**子句**。

例如， $P(x) \vee Q(x)$ ， $P(x, f(x)) \vee Q(x, g(x))$ 都是子句。

◆ **定义12** 不含任何文字的子句称为**空子句**。

由于空子句不含有任何文字，也就不能被任何解释所满足，因此空子句是永假的，不可满足的。空子句一般被记为 $\square$ 或NIL。

◆ **定义13** 由子句或空子句所构成的集合称为**子句集**。

3.4.2 子句集及其应用

□ 子句集的化简

在谓词逻辑中，任何一个谓词公式都可以通过应用等价关系及推理规则化成相应的子句集。

例如公式

$$(\forall x)((\forall y)P(x, y) \rightarrow \neg(\forall y)(Q(x, y) \rightarrow R(x, y)))$$

其化简步骤如下：

(1) 消去连接词 “ $\rightarrow$ ” 和 “ $\leftrightarrow$ ”

反复使用如下等价公式：

$$P \rightarrow Q \Leftrightarrow \neg P \vee Q$$

$$P \leftrightarrow Q \Leftrightarrow (P \wedge Q) \vee (\neg P \wedge \neg Q)$$

即可消去谓词公式中的连接词 “ $\rightarrow$ ” 和 “ $\leftrightarrow$ ”。

以上谓词公式经等价变化后为

$$(\forall x)(\neg(\forall y)P(x, y) \vee \neg(\forall y)(\neg Q(x, y) \vee R(x, y)))$$

3.4.2 子句集及其应用

□ 子句集的化简

(2) 减少否定符号的辖域
反复使用双重否定率

$$\neg(\neg P) \Leftrightarrow P$$

摩根定律

$$\neg(P \wedge Q) \Leftrightarrow \neg P \vee \neg Q$$

$$\neg(P \vee Q) \Leftrightarrow \neg P \wedge \neg Q$$

量词转换率

$$\neg(\forall x)P(x) \Leftrightarrow (\exists x) \neg P(x)$$

$$\neg(\exists x)P(x) \Leftrightarrow (\forall x) \neg P(x)$$

将每个否定符号“ $\neg$ ”移到仅靠谓词的位置，使得每个否定符号最多只作用于一个谓词上。

例如，上式经等价变换后为

$$(\forall x)((\exists y)\neg P(x, y) \vee (\exists y)(Q(x, y) \wedge \neg R(x, y)))$$

$$(\forall x)(\neg(\forall y)P(x, y) \vee \neg(\forall y)(\neg Q(x, y) \vee R(x, y)))$$

3.4.2 子句集及其应用

□ 子句集的化简

$$(\forall x)((\exists y)\neg P(x, y) \vee (\exists y)(Q(x, y) \wedge \neg R(x, y)))$$

(3) 对变元标准化

在一个量词的辖域内，把谓词公式中受该量词约束的变元全部用另外一个没有出现过的任意变元代替，使不同量词约束的变元有不同的名字。

例如，上式经变换后为

$$(\forall x)((\exists y)\neg P(x, y) \vee (\exists z)(Q(x, z) \wedge \neg R(x, z)))$$

(4) 化为前束范式

化为前束范式的方法：把所有量词都移到公式的左边，并且在移动时**不能改变其相对顺序**。由于第(3)步已对变元进行了标准化，每个量词都有自己的变元，这就消除了任何由变元引起冲突的可能，因此这种移动是可行的。

例如，上式化为前束范式后为

$$(\forall x)(\exists y)(\exists z)(\neg P(x, y) \vee (Q(x, z) \wedge \neg R(x, z)))$$

3.4.2 子句集及其应用

□ 子句集的化简

(5) 消去存在量词 $(\forall x)(\exists y)(\exists z)(\neg P(x, y) \vee (Q(x, z) \wedge \neg R(x, z)))$

消去存在量词时，需要区分以下两种情况：

若存在量词不出现在全称量词的辖域内（即它的左边没有全称量词），只要用一个新的个体常量替换受该存在量词约束的变元，就可消去该存在量词。

若存在量词位于一个或多个全称量词的辖域内，例如

$$(\forall x_1) \dots (\forall x_n) (\exists y) P(x_1, x_2, \dots, x_n, y)$$

则需要用Skolem函数 $f(x_1, x_2, \dots, x_n)$ 替换受该存在量词约束的变元 y ，然后再消去该存在量词。

例如，上步所得公式中存在量词 $(\exists y)$ 和 $(\exists z)$ 都位于 $(\forall x)$ 的辖域内，因此都需要用Skolem函数来替换。设替换 y 和 z 的Skolem函数分别是 $f(x)$ 和 $g(x)$ ，则替换后的式子为

$$(\forall x)(\neg P(x, f(x)) \vee (Q(x, g(x)) \wedge \neg R(x, g(x))))$$

(6) 化为Skolem标准形

对上述前束范式的母式应用以下等价关系

$$P \vee (Q \wedge R) \Leftrightarrow (P \vee Q) \wedge (P \vee R)$$

例如，前面的公式化为Skolem标准形后为

$$(\forall x)(\neg P(x, f(x)) \vee (Q(x, g(x)) \wedge (\neg P(x, f(x)) \vee \neg R(x, g(x))))$$

3.4.2 子句集及其应用

□ 子句集的化简

(7) 消去全称量词 $(\forall x)(\neg P(x, f(x)) \vee (Q(x, g(x)) \wedge (\neg P(x, f(x)) \vee \neg R(x, g(x))))$

由于母式中的全部变元均受全称量词约束，并且与全称量词的次序无关，因此可省掉全称量词。但剩下的母式，仍假设其变元是被全称量词量化的。

例如，上式消去全称量词后为

$$(\neg P(x, f(x)) \vee Q(x, g(x)) \wedge (\neg P(x, f(x)) \vee \neg R(x, g(x))))$$

(8) 消去合取词

在母式中消去所有合取词，把母式用子句集的形式表示出来。其中，子句集中的每一个元素都是一个子句。

例如，上式的子句集中包含以下两个子句

$$\neg P(x, f(x)) \vee Q(x, g(x))$$

$$\neg P(x, f(x)) \vee \neg R(x, g(x))$$

(9) 更换变量名称

对子句集中的某些变量重新命名，使任意两个子句中不出现相同的变量名。由于任意两个不同子句的变量之间实际上不存在任何关系。这样，更换变量名是不会影响公式的真值的。

例如，对前式，可把第二个子句集中的变元 x 更换为 y ，得到如下子句集

$$\neg P(x, f(x)) \vee Q(x, g(x))$$

$$\neg P(y, f(y)) \vee \neg R(y, g(y))$$

3.4.2 子句集及其应用

□ 子句集的应用

子句集化简过程的唯一性及其对不可满足性的影响

由于子句集化简过程在消去存在量词时所用的Skolem函数可以不同，因此所得到的**标准子句集不唯一**。

当原谓词公式为**可满足**时，它与其标准子句集**不一定等价**。但当原谓词公式为**不可满足**时，其标准子句集则一定是**不可满足的**，即Skolem化并不影响原谓词公式的**不可满足性**。

这个结论很重要，是归结原理的主要依据，可用定理的形式来描述。

定理3.1 设有谓词公式 F ，其标准子句集为 S ，则 F 为不可满足的充要条件是 S 为不可满足的。

3.4.2 子句集及其应用

□ 子句集的应用

由于连词化归律、双重否定律、狄摩根律、量词转化律、变元标准化得到的均为等价式，因此，F化为前束形之后与谓词公式F是等价的。另外，消去存在量词以后的各个步骤也均使用等价式，所以语义不会发生变化。

由此可见，**不等价只出现在消去存在量词那个步骤。**

为证明此定理，先作如下说明：

为讨论问题方便，设给定的谓词公式F已为前束形

$$(Q_1 x_1) \dots (Q_r x_r) \dots (Q_n x_n) M(x_1, x_2, \dots, x_n)$$

其中， $M(x_1, x_2, \dots, x_n)$ 已化为合取范式。

由于将F化为这种前束形是一种很容易实现的等价运算，因此这种假设是可以的。

3.4.2 子句集及其应用

□ 子句集的应用

又设 $(Q_r x_r)$ 是第一个出现的存在量词 $(\exists x_r)$, 即F为

$$F=(\forall x_1) \dots (\forall x_{r-1}) (\exists x_r) (Q_{r+1} x_{r+1}) \dots (Q_n x_n) M(x_1, \dots, x_{r-1}, x_r, x_{r+1}, \dots, x_n)$$

为把F化为Skolem形, 需要先消去这个 $(\exists x_r)$, 并引入Skolem函数, 得到

$$F_1=(\forall x_1) \dots (\forall x_{r-1}) (Q_{r+1} x_{r+1}) \dots (Q_n x_n) M(x_1, \dots, x_{r-1}, f(x_1, \dots, x_{r-1}), x_{r+1}, \dots, x_n)$$

若能证明

$$F \text{ 不可满足} \Leftrightarrow F_1 \text{ 不可满足}$$

则同理可证

$$F_1 \text{ 不可满足} \Leftrightarrow F_2 \text{ 不可满足}$$

重复这一过程, 直到证明了

$$F_{m-1} \text{ 不可满足} \Leftrightarrow F_m \text{ 不可满足}$$

为止。

此时, F_m 已为F的Skolem标准形。而S只不过是 F_m 的一种集合表示形式。因此有
 $F_m \text{ 不可满足} \Leftrightarrow S \text{ 不可满足}$

3.4.2 子句集及其应用

□ 子句集的应用

下面开始用反证法证明

证明: F 不可满足 $\Leftrightarrow F_1$ 不可满足

证明 $\Rightarrow$

已知 F 不可满足, 假设 F_1 是可满足的, 则存在一个解释 I , 使 F_1 在解释 I 下为真。即对任意 $x_1, \dots, x_{r-1}$ 在 I 的设定下有

$$(Q_{r+1} x_{r+1}) \cdots (Q_n x_n) M(x_1, \dots, x_{r-1}, f(x_1, \dots, x_{r-1}), x_{r+1}, \dots, x_n)$$

为真。亦即对任意的 $x_1, \dots, x_{r-1}$ 都有一个 $f(x_1, \dots, x_{r-1})$ 使

$$(Q_{r+1} x_{r+1}) \cdots (Q_n x_n) M(x_1, \dots, x_{r-1}, f(x_1, \dots, x_{r-1}), x_{r+1}, \dots, x_n)$$

为真。即在 I 下有

$$(\forall x_1) \cdots (\forall x_{r-1}) (\exists x_r) (Q_{r+1} x_{r+1}) \cdots (Q_n x_n) M(x_1, \dots, x_{r-1}, x_r, x_{r+1}, \dots, x_n)$$

为真。即 F 在 I 下为真。

但这与前提 F 是不可满足的相矛盾, 即假设 F_1 为可满足是错误的。从而可以得出“若 F 不可满足, 则必有 F_1 不可满足”。

3.4.3 鲁滨逊归结原理

□ 子句集的应用

再证明 $\Leftarrow$

已知 F_1 不可满足, 假设 F 是可满足的。于是便有某个解释 I 使 F 在 I 下为真。即对任意的 $x_1, \dots, x_{r-1}$ 在 I 的设定下都可找到一个 x_r 使

$$(Q_{r+1} x_{r+1}) \cdots (Q_n x_n) M(x_1, \dots, x_{r-1}, x_r, x_{r+1}, \dots, x_n)$$

为真。若扩充 I , 使它包含一个函数 $f(x_1, \dots, x_{r-1})$, 且有

$$x_r = f(x_1, \dots, x_{r-1})$$

这样, 就可以把所有的 $f(x_1, \dots, x_{r-1})$ 映射到 x_r , 从而得到一个新的解释 I' , 并且在此解释下对任意的 $x_1, \dots, x_{r-1}$ 都有

$$(Q_{r+1} x_{r+1}) \cdots (Q_n x_n) M(x_1, \dots, x_{r-1}, f(x_1, \dots, x_{r-1}), x_{r+1}, \dots, x_n)$$

为真。即在 I' 下有

$$(\forall x_1) \cdots (\forall x_{r-1}) (Q_{r+1} x_{r+1}) \cdots (Q_n x_n) M(x_1, \dots, x_{r-1}, f(x_1, \dots, x_{r-1}), x_{r+1}, \dots, x_n)$$

为真。它说明 F_1 在解释 I' 下为真。但这与前提 F_1 是不可满足的相矛盾, 即**假设 F 为可满足是错误的**。从而可以得出“若 F_1 不可满足, 则必有 F 不可满足”

于是, 定理得证。

由此定理可知, 要证明一个谓词公式是不可满足的, 只要证明其相应的标准子句集是不可满足的就可以了。至于如何证明一个子句集的不可满足性, 由下面的和鲁滨逊归结原理来解决。

3.4.3 鲁滨逊归结原理

□ 基本思想

➤ 注意以下两个关键

第一，子句集中的子句之间是**合取**关系。因此，子句集中只要有一个子句为不可满足，则整个子句集就是不可满足的；

第二，空子句是不可满足的。因此，一个子句集中如果包含有空子句，则此子句集就一定是不可满足的。

➤ 鲁滨逊归结原理基本思想

首先把欲证明问题的结论否定，并加入子句集，得到一个扩充的子句集 S' 。然后设法检验子句集 S' 是否含有空子句，若含有空子句，则表明 S' 是不可满足的；若不含有空子句，则继续使用归结法，在子句集中选择合适的子句进行归结，直至导出空子句或不能继续归结为止。

➤ 鲁滨逊归结原理包括

命题逻辑归结原理

谓词逻辑归结原理

3.4.3 鲁滨逊归结原理

□ 命题逻辑的归结

归结推理的核心是求两个子句的归结式

(1) 归结式的定义及性质

定义23 若 P 是原子谓词公式，则称 P 与 $\neg P$ 为**互补文字**。

定义24 设 C_1 和 C_2 是子句集中的任意两个子句，如果 C_1 中的文字 L_1 与 C_2 中的文字 L_2 互补，那么可从 C_1 和 C_2 中分别消去 L_1 和 L_2 ，并将 C_1 和 C_2 中余下的部分按析取关系构成一个新的子句 C_{12} ，则称这一过程为**归结**，称 C_{12} 为 C_1 和 C_2 的**归结式**，称 C_1 和 C_2 为 C_{12} 的**亲本子句**。

例 设 $C_1 = P \vee Q \vee R$ ， $C_2 = \neg P \vee S$ ，求 C_1 和 C_2 的归结式 C_{12} 。

例 设 $C_1 = \neg Q$ ， $C_2 = Q$ ，求 C_1 和 C_2 的归结式 C_{12} 。

3.4.3 鲁滨逊归结原理

□ 命题逻辑的归结

例 设 $C_1 = \neg P \vee Q$, $C_2 = \neg Q$, $C_3 = P$,
求 C_1 、 C_2 、 C_3 的归结式 C_{123} 。

解：若先对 C_1 、 C_2 归结，可得到

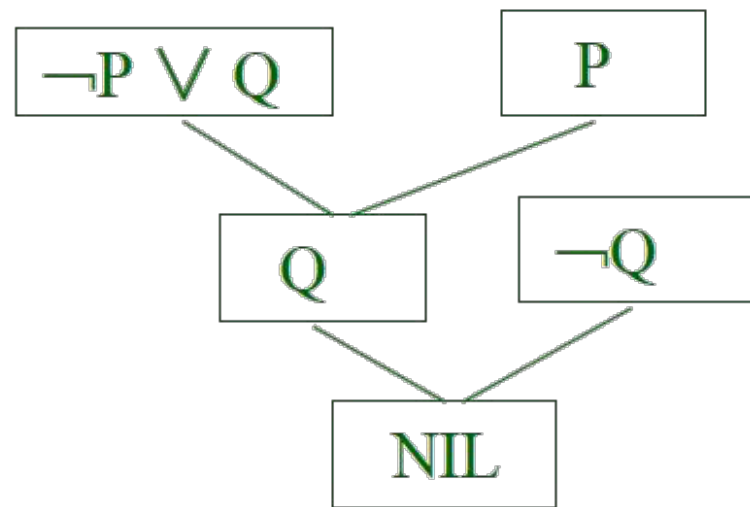
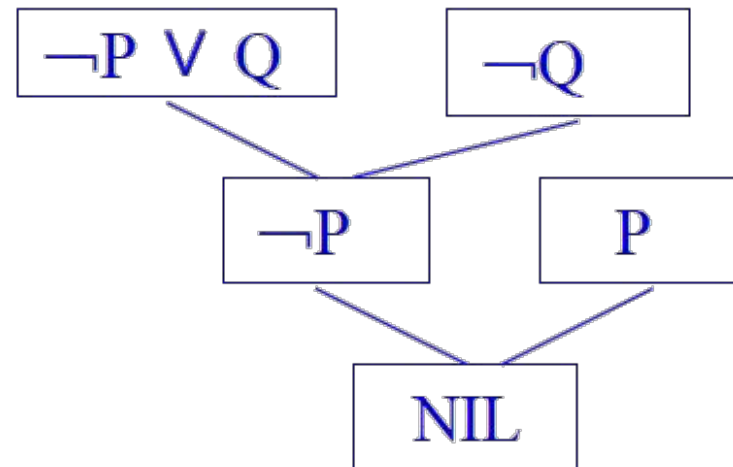
$$C_{12} = \neg P$$

然后再对 C_{12} 和 C_3 归结，得到

$$C_{123} = \text{NIL}$$

如果改变归结顺序，同样可以得到相同的结果，即其归结过程是不唯一的。

其归结过程可用右图来表示，该树称为
归结树。



3.4.3 鲁滨逊归结原理

□ 命题逻辑的归结

定理3.7 归结式 C_{12} 是其亲本子句 C_1 和 C_2 的逻辑结论。

证明：（按定义）设 $C_1 = L \vee C_1'$ ， $C_2 = \neg L \vee C_2'$ 关于解释 I 为真，则只需证明 $C_{12} = C_1' \vee C_2'$ 关于解释 I 也为真。

对于解释 I ， L 和 $\neg L$ 中必有一个为假。

若 L 为假，则必有 C_1' 为真，不然就会使 C_1 为假，这将与前提假设 C_1 为真矛盾，因此只能有 C_1' 为真。

同理，若 $\neg L$ 为假，则必有 C_2' 为真。

因此，必有 $C_{12} = C_1' \vee C_2'$ 关于解释 I 也为真。即 C_{12} 是 C_1 和 C_2 的逻辑结论。

另证： $C_1: \neg L \rightarrow C_1'$ ， $C_2: L \rightarrow C_2' \Rightarrow C_{12} = C_1' \vee C_2'$

3.4.3 鲁滨逊归结原理

□ 命题逻辑的归结

上述定理是归结原理中的一个重要定理，由它可得到以下两个推论：

推论1： 设 C_1 和 C_2 是子句集 S 中的两个子句， C_{12} 是 C_1 和 C_2 的归结式，若用 C_{12} 代替 C_1 和 C_2 后得到新的子句集 S_1 ，则由 S_1 的不可满足性可以推出原子句集 S 的不可满足性。即：

S_1 的不可满足性 $\Rightarrow S$ 的不可满足性

证明： 设 $S=\{C_1, C_2, C_3, \dots, C_n\}$ ， C_{12} 是 C_1 和 C_2 的归结式，则用 C_{12} 代替 C_1 和 C_2 后可得到一个新的子句集

$$S_1 = \{C_{12}, C_3, \dots, C_n\}$$

设 S_1 是不可满足的，则对不满足 S_1 的任一解释 I ，都可能有以下两种情况：

①解释 I 使 C_{12} 为真，则 $C_3, \dots, C_n$ 中必有一个为假，即 S 是不可满足的。

②解释 I 使 C_{12} 为假，即 $\neg C_{12}$ 为真， $\neg(C_1 \wedge C_2)$ 永真，即 $\neg C_1 \vee \neg C_2$ 永真，它说明解释 I 使 C_1 为假，或 C_2 为假。即 S 也是不可满足的。

$(C_1 \wedge C_2 \rightarrow C_{12})$

因此可以得出

S_1 的不可满足性 $\Rightarrow S$ 的不可满足性

3.4.3 鲁滨逊归结原理

□ 命题逻辑的归结

推论2：设 $C1$ 和 $C2$ 是子句集 S 中的两个子句， $C12$ 是 $C1$ 和 $C2$ 的归结式，若把 $C12$ 加入 S 中得到新的子句集 $S2$ ，则 S 与 $S2$ 的不可满足性是等价的。即：

$S2$ 的不可满足性 $\Leftrightarrow S$ 的不可满足性

先证明 $\Leftarrow$

设 $S=\{ C1 , C2 , C3 , \dots, Cn \}$ 是不可满足的，则 $C1 , C2 , C3 , \dots, Cn$ 中必有一子句为假，因而 $S2 =\{ C12 , C1 , C2 , C3 , \dots, Cn \}$ 必为不可满足。

再证明 $\Rightarrow$

设 $S2$ 是不可满足的,则对不满足 $S2$ 的任一解释 I ,都可能有以下两种情况：

①解释 I 使 $C12$ 为真，则 $C1 , C2 , C3 , \dots, Cn$ 中必有一个为假，即 S 是不可满足的。

②解释 I 使 $C12$ 为假，即 $\neg C12$ 为真， $\neg(C1 \wedge C2)$ 永真，即 $\neg C1 \vee \neg C2$ 永真，它说明解释 I 使 $C1$ 为假，或 $C2$ 为假。即 S 也是不可满足的。

由此可见 S 与 $S2$ 的不可满足性是等价的。即 S 的不可满足性 $\Leftrightarrow S2$ 的不可满足性

3.4.3 鲁滨逊归结原理

□ 命题逻辑的归结

上述两个推论说明，为证明子句集 S 的不可满足性，只要对其中可进行归结的子句进行归结，并把归结式加入到子句集 S 中，或者用归结式代替他的亲本子句，然后对新的子句集证明其不可满足性就可以了。

如果经归结能得到空子句，根据空子句的不可满足性，即可得到原子句集 S 是不可满足的结论。

在命题逻辑中，对不可满足的子句集 S ，其归结原理是完备的。

这种不可满足性可用如下定理描述：

定理3.8 子句集 S 是不可满足的，当且仅当存在一个从 S 到空子句的归结过程。

3.4.3 鲁滨逊归结原理

□ 谓词逻辑的归结

在谓词逻辑中，由于子句集中的谓词一般都含有变元，因此不能像命题逻辑那样直接消去互补文字。而需要先用变元置换对谓词进行合一，然后才能进行归结。可见，谓词逻辑的归结要比命题逻辑的归结麻烦一些。

谓词逻辑的归结原理

谓词逻辑中的归结式可用如下定义来描述：

定义25 设C1和C2是两个没有公共变元的子句，L1和L2分别是C1和C2中的文字。如果L1和 $\neg L2$ 存在一个最一般合一置换 σ ，则称

$$C12 = (\{C1\sigma\} - \{L1\sigma\}) \cup (\{C2\sigma\} - \{L2\sigma\})$$

为C1和C2的二元归结式，而L1和L2为归结式上的文字。

3.4.3 鲁滨逊归结原理

□ 谓词逻辑的归结

对以上讨论做以下两点说明：

- (1) 这里之所以使用集合符号和集合的运算，目的是为了说明问题的方便。即先将子句 C_i 和 L_i 写成集合的形式，在集合表示下做减法和并集运算，然后再写成子句集的形式。
- (2) 定义中还要求 C_1 和 C_2 无公共变元，这也是合理的。

例如 $C_1 = P(x)$ ， $C_2 = \neg P(f(x))$ ，而 $S = \{ C_1, C_2 \}$ 是不可满足的。但由于 C_1 和 C_2 的变元相同，就无法合一了。没有归结式，就不能用归结法证明 S 的不可满足性，这就限制了归结法的使用范围。

如果对 C_1 或 C_2 的变元进行换名，便可通过合一，对 C_1 和 C_2 进行归结。如上例，若先对 C_2 进行换名，即 $C_2 = \neg P(f(y))$ ，则可对 C_1 和 C_2 进行归结，得到一个空子句，从而证明了 S 是不可满足的。

事实上，在由公式集化为子句集的过程中，其最后一步就是做换名处理。因此，定义中假设 C_1 和 C_2 没有相同变元是可以的。

3.4.3 鲁滨逊归结原理

□ 谓词逻辑的归结

例 设 $C_1 = P(a) \vee R(x)$, $C_2 = \neg P(y) \vee Q(b)$, 求 C_{12}

解: 取 $L_1 = P(a)$, $L_2 = \neg P(y)$, 则 L_1 和 L_2 的合一置换是 $\sigma = \{a/y\}$ 。根据定义可得

$$\begin{aligned} C_{12} &= (\{C_1 \sigma\} - \{L_1 \sigma\}) \cup (\{C_2 \sigma\} - \{L_2 \sigma\}) \\ &= (\{P(a), R(x)\} - \{P(a)\}) \cup (\{\neg P(a), Q(b)\} - \{\neg P(a)\}) \\ &= (\{R(x)\}) \cup (\{Q(b)\}) = \{R(x), Q(b)\} \\ &= R(x) \vee Q(b) \end{aligned}$$

例 设 $C_1 = P(x) \vee Q(a)$, $C_2 = \neg P(b) \vee R(x)$, 求 C_{12}

解: 由于 C_1 和 C_2 有相同的变元 x , 不符合谓词逻辑归结定义的要求。为了进行归结, 需要修改 C_2 中变元 x 的名字为 y , 令 $C_2 = \neg P(b) \vee R(y)$ 。此时 $L_1 = P(x)$, $L_2 = \neg P(b)$, L_1 和 L_2 的合一置换是 $\sigma = \{b/x\}$ 。则有

$$\begin{aligned} C_{12} &= (\{C_1 \sigma\} - \{L_1 \sigma\}) \cup (\{C_2 \sigma\} - \{L_2 \sigma\}) \\ &= (\{P(b), Q(a)\} - \{P(b)\}) \cup (\{\neg P(b), R(y)\} - \{\neg P(b)\}) \\ &= (\{Q(a)\}) \cup (\{R(y)\}) = \{Q(a), R(y)\} \\ &= Q(a) \vee R(y) \end{aligned}$$

3.4.3 鲁滨逊归结原理

□ 谓词逻辑的归结

例 设 $C_1 = P(x) \vee \neg Q(b)$, $C_2 = \neg P(a) \vee Q(y) \vee R(z)$

解：对 C_1 和 C_2 通过合一置换 $\{a/x, b/y\}$ 的作用，可以得到两个互补对。

注意：求归结式不能同时消去两个互补对，其结果不是二元归结式。如在 $\sigma = \{a/x, b/y\}$ 下，若同时消去两个互补对所得 $R(z)$ 不是 C_1 和 C_2 的二元归结式。

例 设 $C_1 = P(x) \vee P(f(a)) \vee Q(x)$, $C_2 = \neg P(y) \vee R(b)$, 求 C_{12}

解：对参加归结的某个子句，若其内部有可合一的文字，则在进行归结之前应首先进行合一。本例 C_1 中有 $P(x)$ 与 $P(f(a))$ ，若它用们的合一置换 $\sigma = \{f(a)/x\}$ 进行代换，可得到

$$C_1 \sigma = P(f(a)) \vee Q(f(a))$$

此时可对 $C_1 \sigma$ 与 C_2 进行归结。选 $L_1 = P(f(a))$, $L_2 = \neg P(y)$, L_1 和 L_2 的合一是 $\sigma' = \{f(a)/y\}$ ，则可得到 C_1 和 C_2 的二元归结式为

$$C_{12} = R(b) \vee Q(f(a))$$

其中， $C_1 \sigma$ 为 C_1 的因子。若 C 中有两个或两个以上的文字具有合一置换 σ ，则称 $C\sigma$ 为子句 C 的因子。若 $C\sigma$ 是一个单文字，则称它为 C 的单元因子。

应用因子概念，可对谓词逻辑中的归结原理给出如下定义：

3.4.3 鲁滨逊归结原理

□ 谓词逻辑的归结

定义26 若 C_1 和 C_2 是无公共变元的子句，则

- ① C_1 和 C_2 的二元归结式；
- ② C_1 和 C_2 的因子 $C_1 \sigma_1$ 的二元归结式；
- ③ C_1 的因子 $C_1 \sigma_1$ 和 C_2 的二元归结式；
- ④ C_1 的因子 $C_1 \sigma_1$ 和 C_2 的因子 $C_2 \sigma_2$ 的二元归结式。

这四种二元归结式都是子句 C_1 和 C_2 的二元归结式，记为 C_{12} 。

例 设 $C_1 = P(y) \vee P(f(x)) \vee Q(g(x))$ ， $C_2 = \neg P(f(g(a))) \vee Q(b)$ ，求 C_{12} 。

解：对 C_1 ，取合一 $\sigma = \{f(x)/y\}$ ，得 C_1 的因子

$$C_1 \sigma = P(f(x)) \vee Q(g(x))$$

对 C_1 的因子和 C_2 归结（ $\sigma = \{g(a)/x\}$ ），可得到 C_1 和 C_2 的二元归结式

$$C_{12} = Q(g(g(a))) \vee Q(b)$$

说明：对谓词逻辑，定理3.7仍然适用，即归结式 C_{12} 是其亲本子句 C_1 和 C_2 的逻辑结论。用归结式取代它在子句集 S 中的亲本子句，所得到的子句集仍然保持着原子句集 S 的不可满足性。

此外，对谓词逻辑定理3.8也仍然适用，即从不可满足的意义上说，一阶谓词逻辑的归结原理也是完备的。

3.4.4 归结演绎推理的方法

□ 命题逻辑的归结反演

归结原理

假设 F 为已知前提， G 为欲证明的结论，归结原理把证明 G 为 F 的逻辑结论转化为证明 $F \wedge \neg G$ 为不可满足。

再根据定理3.1，在不可满足的意义上，公式集 $F \wedge \neg G$ 与其子句集是等价的，即把公式集上的不可满足转化为子句集上的不可满足。

归结反演

应用归结原理证明定理的过程称为归结反演。

归结反演过程

在命题逻辑中，已知 F ，证明 G 为真的归结反演过程如下：

- ①否定目标公式 G ，得 $\neg G$ ；
- ②把 $\neg G$ 并入到公式集 F 中，得到 $\{F, \neg G\}$ ；
- ③把 $\{F, \neg G\}$ 化为子句集 S 。
- ④应用归结原理对子句集 S 中的子句进行归结，并把每次得到的归结式并入 S 中。如此反复进行，若出现空子句，则停止归结，此时就证明了 G 为真。

3.4.4 归结演绎推理的方法

□ 命题逻辑的归结反演

例 设已知的公式集为 $\{P, (P \wedge Q) \rightarrow R, (S \vee T) \rightarrow Q, T\}$, 求证结论 R

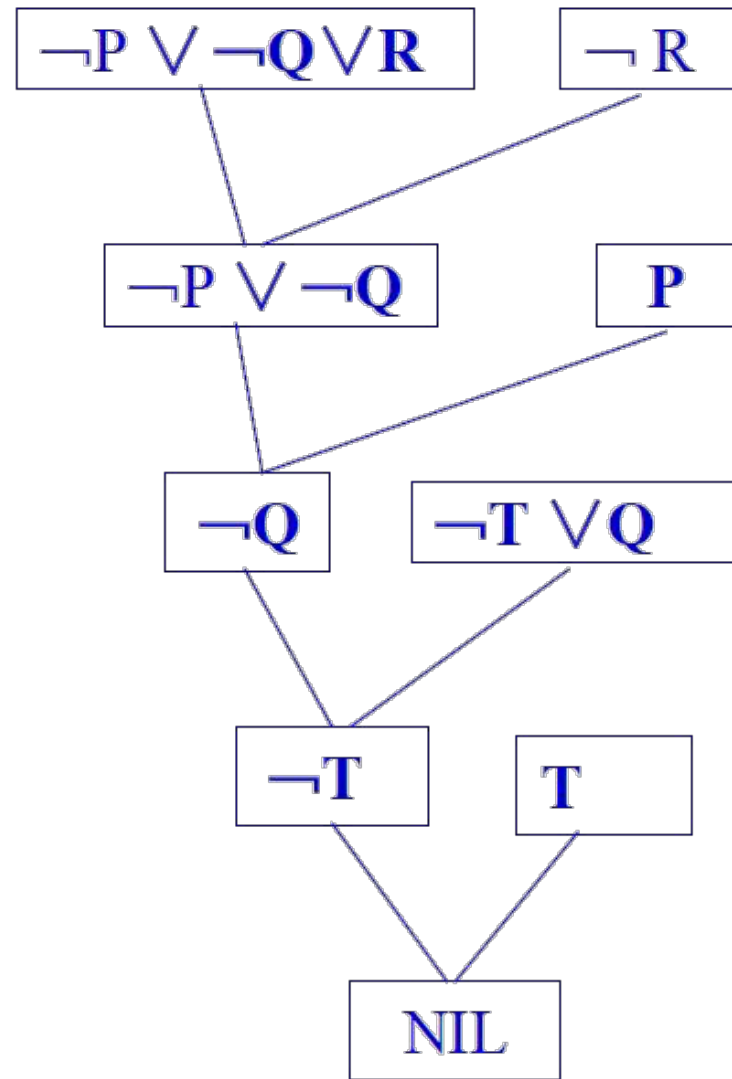
解: 假设结论 R 为假, 将 $\neg R$ 加入公式集, 并化为子句集

$S = \{P, \neg P \vee \neg Q \vee R, \neg S \vee Q, \neg T \vee Q, T, \neg R\}$

其归结过程如右图的归结树所示。

其含义为: 利用归结原理, 对子句集进行归结, 并把所得的归结式并入子句集中; 重复这一过程, 最后归结出了空子句。

根据归结原理的完备性, 可知子句集 S 是不可满足的, 即 $\neg R$ 为假, 这就证明了 R 为真。



3.4.4 归结演绎推理的方法

□ 谓词逻辑的归结演绎推理

谓词逻辑的归结反演

谓词逻辑的归结反演过程与命题逻辑的归结反演过程相比，其步骤基本相同，但每步的处理对象不同。例如，在步骤(3)化简子句集时，谓词逻辑需要把由谓词构成的公式集化为子句集；在步骤(4)按归结原理进行归结时，谓词逻辑的归结原理需要考虑两个亲本子句的合一。

例 已知

$$F: (\forall x)((\exists y)(A(x, y) \wedge B(y)) \rightarrow (\exists y)(C(y) \wedge D(x, y)))$$

$$G: \neg(\exists x)C(x) \rightarrow (\forall x)(\forall y)(A(x, y) \rightarrow \neg B(y))$$

求证G是F的逻辑结论。

证明：先把G否定，并放入F中，得到的 $\{F, \neg G\}$ 为

$$\{(\forall x)((\exists y)(A(x, y) \wedge B(y)) \rightarrow (\exists y)(C(y) \wedge D(x, y))), \\ \neg(\neg(\exists x)C(x) \rightarrow (\forall x)(\forall y)(A(x, y) \rightarrow \neg B(y)))\}$$

3.4.4 归结演绎推理的方法

□ 谓词逻辑的归结演绎推理

将公式集 $\{F, \neg G\} = \{(\forall x)((\exists y)(A(x,y) \wedge B(y)) \rightarrow (\exists y)(C(y) \wedge D(x,y))),$
 $\neg(\neg(\exists x)C(x) \rightarrow (\forall x)(\forall y)(A(x,y) \rightarrow \neg B(y)))\}$ 转化成子句集

(1) 消去连接词 “ $\rightarrow$ ” 和 “ $\leftrightarrow$ ”

$F: (\forall x)(\neg((\exists y)(A(x,y) \wedge B(y))) \vee (\exists y)(C(y) \wedge D(x,y)))$

$G: \neg((\exists x)C(x) \vee ((\forall x)(\forall y)(\neg A(x,y) \vee \neg B(y))))$

(2) 减少否定符号的辖域

$F: (\forall x)((\forall y)(\neg A(x,y) \vee \neg B(y)) \vee (\exists y)(C(y) \wedge D(x,y)))$

$G: ((\forall x)\neg C(x) \wedge ((\exists x)(\exists y)(A(x,y) \wedge B(y))))$

(3) 对变元标准化

$F: (\forall x)((\forall y)\neg A(x,y) \vee \neg B(y)) \vee (\exists z)(C(z) \wedge D(x,z))$

$G: ((\forall x)\neg C(x) \wedge ((\exists z)(\exists y)(A(z,y) \wedge B(y))))$

(4) 化为前束范式

$F: (\forall x)(\forall y)(\exists z)((\neg A(x,y) \vee \neg B(y)) \vee (C(z) \wedge D(x,z)))$

$G: (\forall x)(\exists z)(\exists y)(\neg C(x) \wedge A(z,y) \wedge B(y))$

3.4.4 归结演绎推理的方法

□ 谓词逻辑的归结演绎推理

将公式集 $\{F, \neg G\} = \{(\forall x)((\exists y)(A(x,y) \wedge B(y)) \rightarrow (\exists y)(C(y) \wedge D(x,y))),$
 $\neg(\neg(\exists x)C(x) \rightarrow (\forall x)(\forall y)(A(x,y) \rightarrow \neg B(y)))\}$ 转化成子句集

(4) 化为前束范式

$F: (\forall x)(\forall y)(\exists z)((\neg A(x,y) \vee \neg B(y)) \vee (C(z) \wedge D(x,z)))$

$G: (\forall x)(\exists z)(\exists y)(\neg C(x) \wedge A(z,y) \wedge B(y))$

(5) 消去存在量词

$F: (\forall x)(\forall y)((\neg A(x,y) \vee \neg B(y)) \vee (C(f(x)) \wedge D(x,f(x))))$

$G: (\forall x)(\neg C(x) \wedge A(m,n) \wedge B(n))$

(6) 化为Skolem标准形 (合取范式)

$F: (\forall x)(\forall y)((\neg A(x,y) \vee \neg B(y) \vee C(f(x))) \wedge (\neg A(x,y) \vee \neg B(y) \vee D(x,f(x))))$

$G: (\forall x)(\neg C(x) \wedge A(m,n) \wedge B(n))$

(7) 消去全称量词

$F: (\neg A(x,y) \vee \neg B(y) \vee C(f(x))) \wedge (\neg A(x,y) \vee \neg B(y) \vee D(x,f(x)))$

$G: \neg C(x) \wedge A(m,n) \wedge B(n)$

3.4.4 归结演绎推理的方法

□ 谓词逻辑的归结演绎推理

将公式集 $\{F, \neg G\} = \{(\forall x)((\exists y)(A(x,y) \wedge B(y)) \rightarrow (\exists y)(C(y) \wedge D(x,y))),$
 $\neg(\neg(\exists x)C(x) \rightarrow (\forall x)(\forall y)(A(x,y) \rightarrow \neg B(y)))\}$ 转化成子句集

(7) 消去全称量词

$F: \underline{(\neg A(x,y) \vee \neg B(y) \vee C(f(x)))} \wedge \underline{(\neg A(x,y) \vee \neg B(y) \vee D(x,f(x)))}$

$G: \underline{\neg C(x)} \wedge \underline{A(m,n)} \wedge \underline{B(n)}$

(8) 消去合取词

$F: \underline{\neg A(x,y) \vee \neg B(y) \vee C(f(x))}, \neg A(x,y) \vee \neg B(y) \vee D(x,f(x))$

$G: \neg C(x), A(m,n), B(n)$

(9) 更换变量名称

子句集 $\{\underline{\neg A(x,y) \vee \neg B(y) \vee C(f(x))}, \neg A(u,v) \vee \neg B(v) \vee D(u,f(u))$
 $\neg C(z), A(m,n), B(n)\}$

3.4.4 归结演绎推理的方法

□ 谓词逻辑的归结演绎推理

再把 $\{F, \neg G\}$ 化成子句集, 得到

$$(1) \neg A(x,y) \vee \neg B(y) \vee C(f(x))$$

$$(2) \neg A(u,v) \vee \neg B(v) \vee D(u,f(u))$$

$$(3) \neg C(z)$$

$$(4) A(m,n)$$

$$(5) B(n)$$

其中, (1)、(2)是由 F 化出的两个子句, (3)、(4)、(5)是由 $\neg G$ 化出的3个子句。

最后应用谓词逻辑的归结原理对上述子句集进行归结, 其过程为

$$(6) \neg A(x,y) \vee \neg B(y) \text{ 由(1)和(3)归结, 取 } \sigma = \{f(x)/z\}$$

$$(7) \neg B(n) \text{ 由(4)和(6)归结, 取 } \sigma = \{m/x, n/y\}$$

$$(8) \text{NIL} \text{ 由(5)和(7)归结}$$

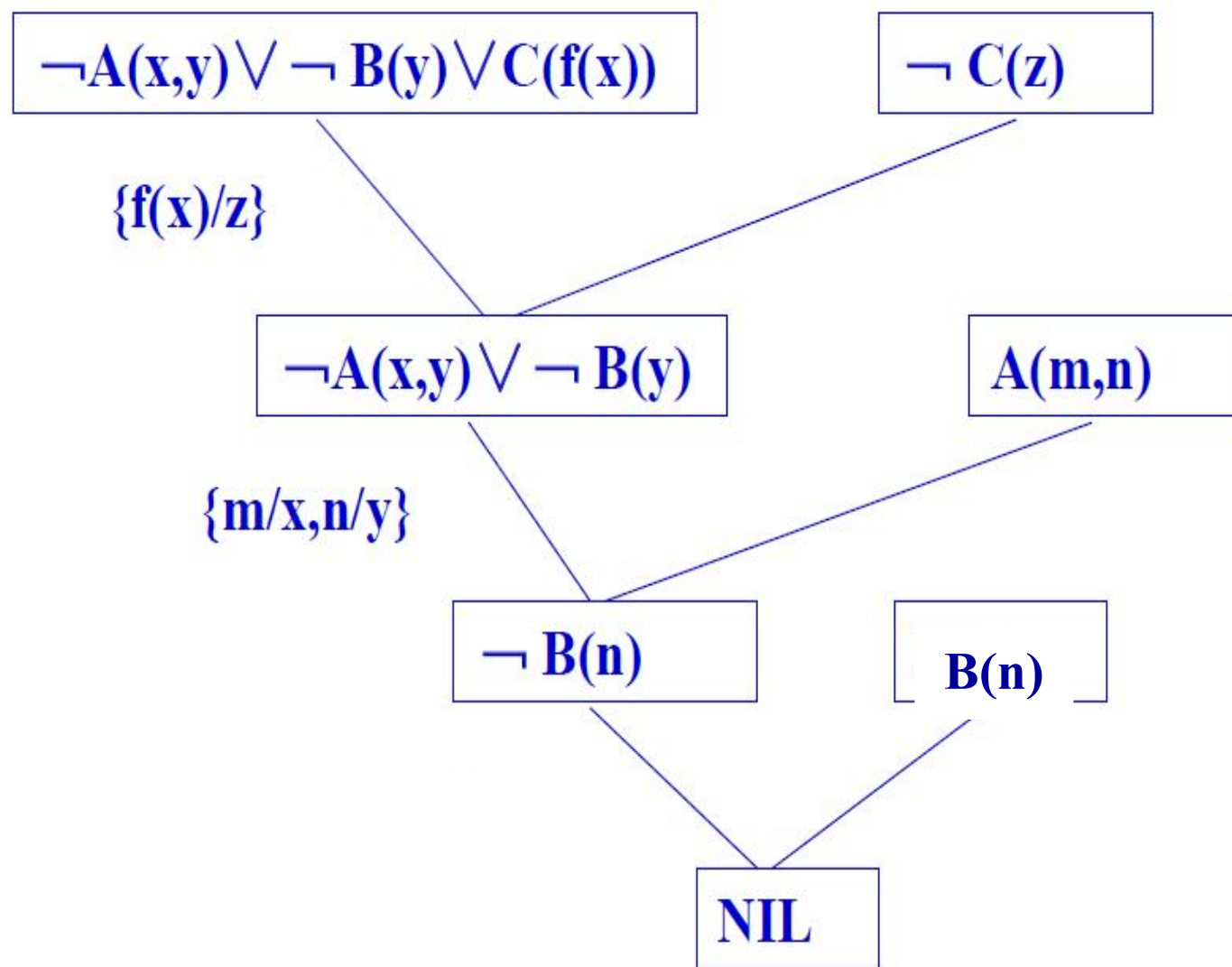
因此, G 是 F 的逻辑结论。

上述归结过程可用如下归结树来表示

$$\begin{aligned} F: & (\forall x)((\exists y)(A(x, y) \wedge B(y)) \rightarrow (\exists y)(C(y) \wedge D(x, y))) \\ G: & \neg((\exists x)C(x) \rightarrow (\forall x)(\forall y)(A(x, y) \rightarrow \neg B(y))) \end{aligned}$$

3.4.4 归结演绎推理的方法

□ 谓词逻辑的归结演绎推理



3.4.4 归结演绎推理的方法

□ 归结演绎推理的经典例子

例 “激动人心的生活” 问题

假设：所有不贫穷并且聪明的人都是快乐的，那些看书的人是聪明的。李明能看书且不贫穷，快乐的人过着激动人心的生活。

求证：李明过着激动人心的生活。

解：先定义谓词：

Poor(x) x是贫穷的

Smart(x) x是聪明的

Happy(x) x是快乐的

Read(x) x能看书

Exciting(x) x过着激动人心的生活

3.4.4 归结演绎推理的方法

□ 归结演绎推理的经典例子

再将问题用谓词表示如下：

“ 所有不贫穷并且聪明的人都是快乐的”

$(\forall x)((\neg \text{Poor}(x) \wedge \text{Smart}(x)) \rightarrow \text{Happy}(x))$

“ 那些看书的人是聪明的”

$(\forall y)(\text{Read}(y) \rightarrow \text{Smart}(y))$

“ 李明能看书且不贫穷”

$\text{Read}(\text{Liming}) \wedge \neg \text{Poor}(\text{Liming})$

“ 快乐的人过着激动人心的生活”

$(\forall z)(\text{Happy}(z) \rightarrow \text{Exciting}(z))$

目标 “ 李明过着激动人心的生活” 的否定
 $\neg \text{Exciting}(\text{Liming})$

3.4.4 归结演绎推理的方法

□ 归结演绎推理的经典例子

将上述谓词公式转化为子句集如下：

$$(1) \text{Poor}(x) \vee \neg \text{Smart}(x) \vee \text{Happy}(x)$$

$$(2) \neg \text{Read}(y) \vee \text{Smart}(y)$$

$$(3) \text{Read}(\text{Liming})$$

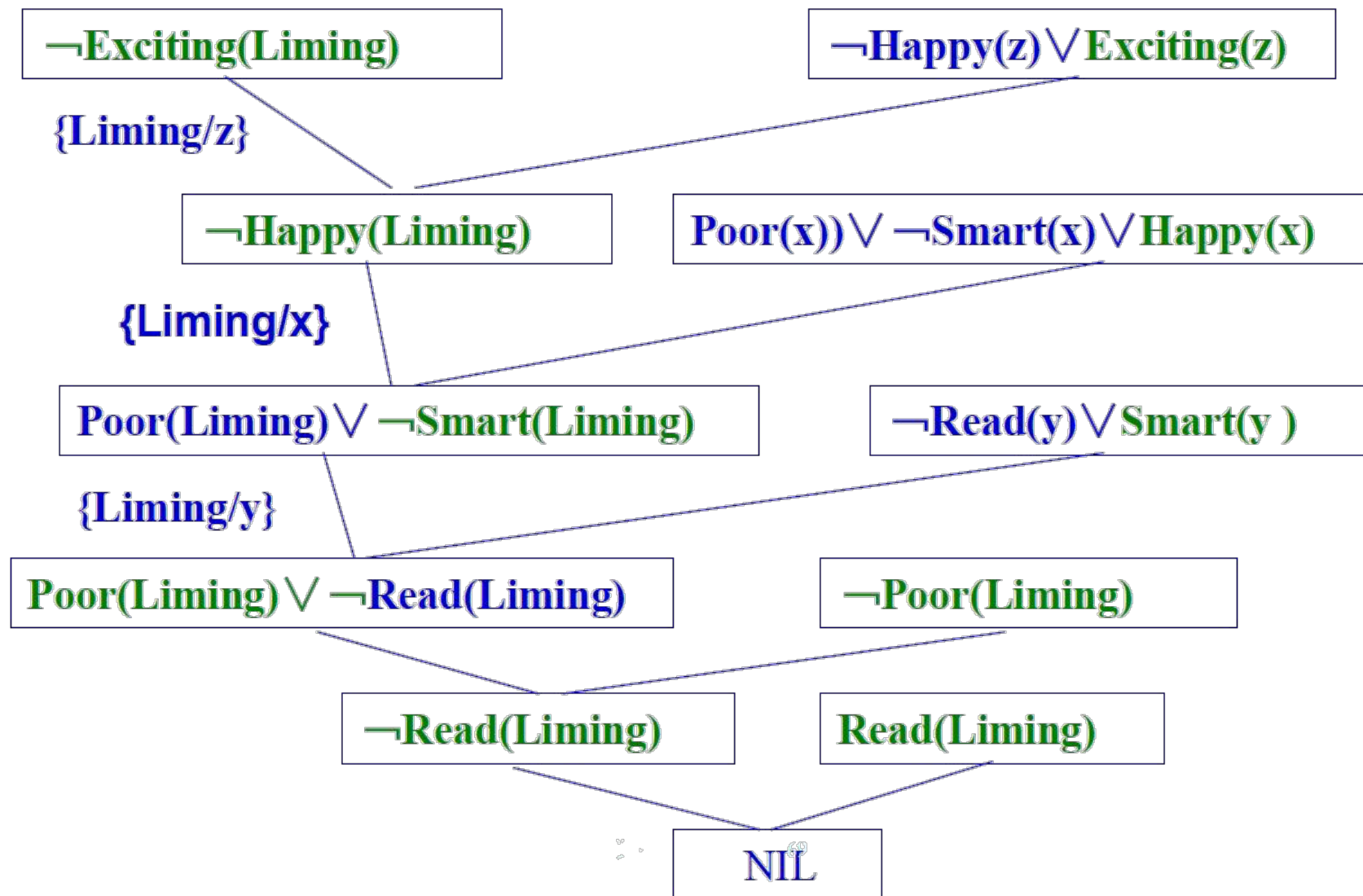
$$(4) \neg \text{Poor}(\text{Liming})$$

$$(5) \neg \text{Happy}(z) \vee \text{Exciting}(z)$$

$$(6) \neg \text{Exciting}(\text{Liming}) \text{ (结论的否定)}$$

3.4.4 归结演绎推理的方法

□ 归结演绎推理的经典例子



3.4.4 归结演绎推理的方法

□ 归结演绎推理的经典例子

下面再给出一个经典的归结问题

例 “快乐学生” 问题

假设：任何通过计算机考试并获奖的人都是快乐的，任何肯学习或幸运的人都可以通过所有考试，张不肯学习但他是幸运的，任何幸运的人都能获奖。

求证：张是快乐的。

解：先定义谓词：Pass(x, y) x可以通过y考试

Win(x, prize) x能获得奖励

Study(x) x肯学习

Happy(x) x是快乐的

Lucky(x) x是幸运的

3.4.4 归结演绎推理的方法

□ 归结演绎推理的经典例子

再将问题用谓词表示如下：

“任何通过计算机考试并获奖的人都是快乐的”

$(\forall x)(\text{Pass}(x, \text{computer}) \wedge \text{Win}(x, \text{prize}) \rightarrow \text{Happy}(x))$

“任何肯学习或幸运的人都可以通过所有考试”

$(\forall x)(\forall y)(\text{Study}(x) \vee \text{Lucky}(x) \rightarrow \text{Pass}(x, y))$

“张不肯学习但他是幸运的”

$\neg \text{Study}(\text{zhang}) \wedge \text{Lucky}(\text{zhang})$

“任何幸运的人都能获奖”

$(\forall x)(\text{Lucky}(x) \rightarrow \text{Win}(x, \text{prize}))$

结论 “张是快乐的” 的否定

$\neg \text{Happy}(\text{zhang})$

3.4.4 归结演绎推理的方法

□ 归结演绎推理的经典例子

将上述谓词公式转化为子句集如下：

$$(1) \neg \text{Pass}(x, \text{computer}) \vee \neg \text{Win}(x, \text{prize}) \vee \text{Happy}(x)$$

$$(2) \neg \text{Study}(y) \vee \text{Pass}(y, z)$$

$$(3) \neg \text{Lucky}(u) \vee \text{Pass}(u, v)$$

$$(4) \neg \text{Study}(\text{zhang})$$

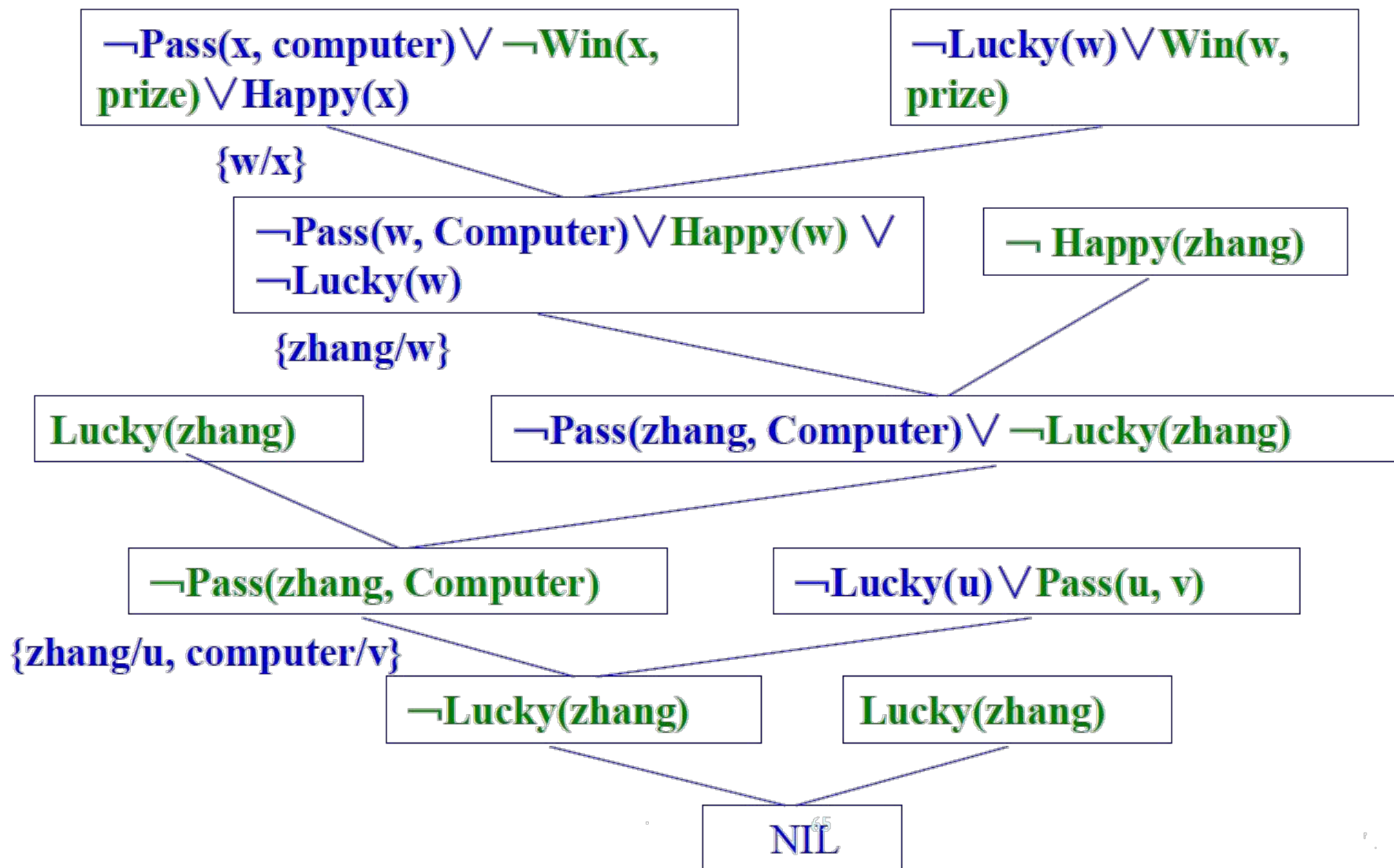
$$(5) \text{Lucky}(\text{zhang})$$

$$(6) \neg \text{Lucky}(w) \vee \text{Win}(w, \text{prize})$$

$$(7) \neg \text{Happy}(\text{zhang}) \text{ (结论的否定)}$$

3.4.4 归结演绎推理的方法

□ 归结演绎推理的经典例子

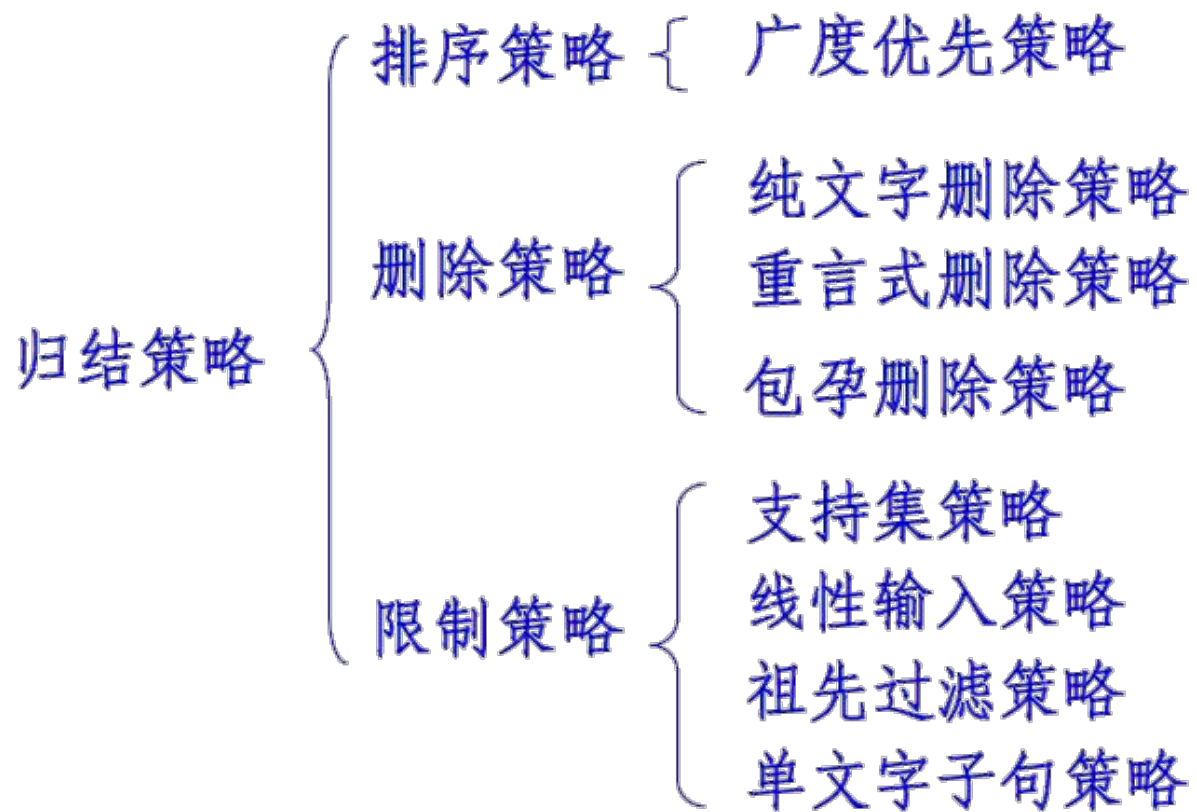


3.4.5 归结演绎推理的归结策略

归结策略是指在归结演绎推理过程的每一步如何选择进行归结的子句对，以尽快得到空子句的策略。

不同的归结策略影响归结过程、归结推理效率和完备性。

目前，常用的归结策略可分为三大类。



3.4.5 归结演绎推理的归结策略

□ 排序策略

广（宽）度优先是一种穷尽子句比较的复杂搜索方法。设初始子句集为 S_0 ，广度优先策略的归结过程可描述如下：

(1) 从 S_0 出发，对 S_0 中的全部子句作所有可能的归结，得到第一层归结式，把这些归结式的集合记为 S_1 ；

(2) 用 S_0 , S_1 中的子句与 S_1 中的子句进行所有可能的归结，得到第二层归结式，把这些归结式的集合记为 S_2 ；

(3) 用 S_0 , S_1 和 S_2 中的子句与 S_2 中的子句进行所有可能的归结，得到第三层归结式，把这些归结式的集合记为 S_3 ；

如此继续，直到得出空子句或不能再继续归结为止。

3.4.5 归结演绎推理的归结策略

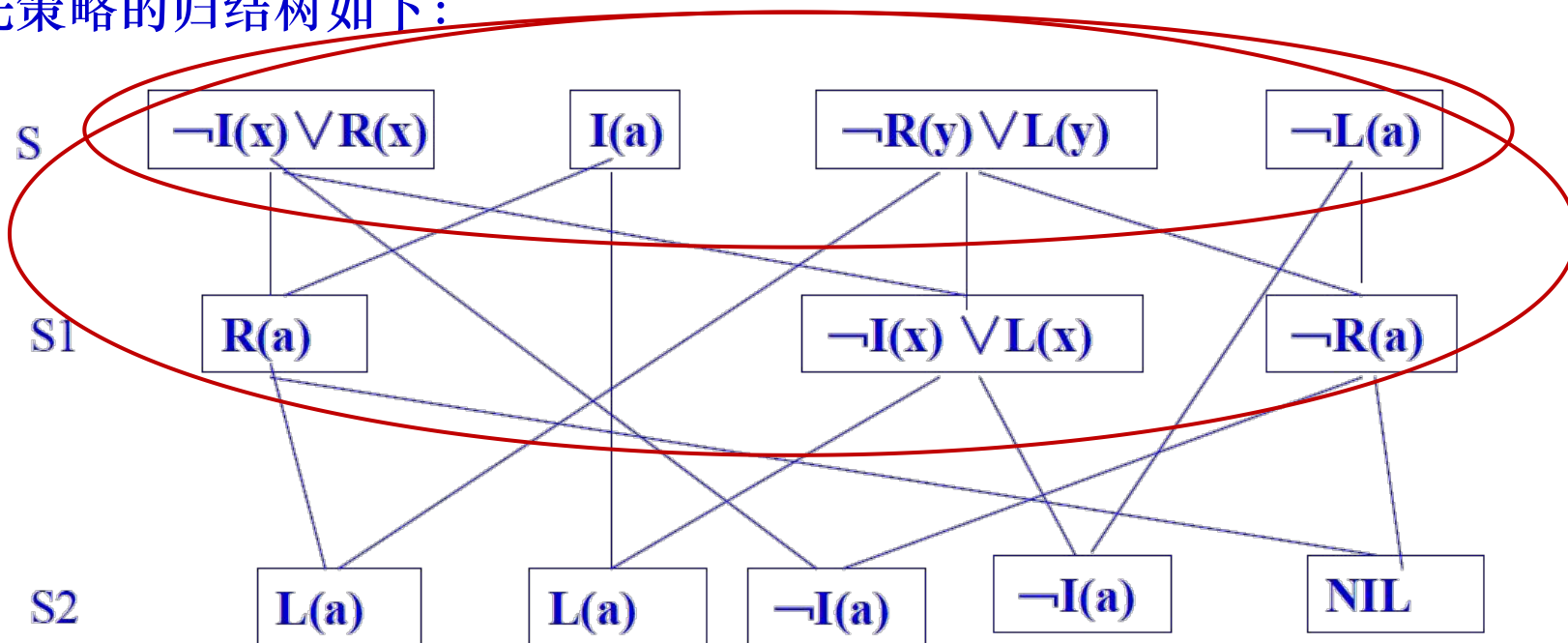
□ 排序策略

例 设有如下子句集:

$$S = \{ \neg I(x) \vee R(x), I(a), \neg R(y) \vee L(y), \neg L(a) \}$$

用广度优先策略证明S为不可满足。

解: 广度优先策略的归结树如下:



广度优先策略是一种穷尽法, 其归结效率较低, 当问题比较复杂时, 还可能会出现组合爆炸。广度优先策略是完备的, 并当问题有解时, 它一定能找到最短归结路径。当问题较小时, 广度优先策略仍然是一种较好的归结策略。

3.4.5 归结演绎推理的归结策略

□ 删除策略

基本思想：归结过程在寻找可归结子句时，子句集中的子句越多，需要付出的代价就会越大。如果在归结时能把子句集中无用的子句删除掉，这就会缩小搜索范围，减少比较次数，从而提高归结效率。

a) 纯文字删除法

纯文字：如果某文字 L 在子句集中不存在可与其互补的文字 $\neg L$ ，则称该文字为纯文字。

在归结过程中，纯文字不可能被消除，用包含纯文字的子句进行归结也不可能得到空子句，因此对包含纯文字的子句进行归结是没有意义的，应该把它从子句集中删除。

对子句集而言，删除包含纯文字的子句，是不影响其不可满足性的。例如，对子句集

$$S = \{P \vee Q \vee R, \neg Q \vee R, Q, \neg R\}$$

其中 P 是纯文字，因此可以将子句 $P \vee Q \vee R$ 从子句集 S 中删除。

3.4.5 归结演绎推理的归结策略

□ 删除策略

b) 重言式删除法

重言式：如果一个子句中包含有互补的文字对，则称其为重言式。如

$$P(x) \vee \neg P(x), P(x) \vee Q(x) \vee \neg P(x)$$

都是重言式。

重言式是真值为真的子句。对于一个子句集来说，不管是增加还是删除一个真值为真的子句，都不会影响该子句集的不可满足性。因此，可删去。

c) 包孕 (含) 删除法

包孕：设有子句 $C1$ 和 $C2$ ，如果存在一个置换 σ ，使得 $C1 \sigma \subseteq C2$ ，则称 $C1$ 包孕于 $C2$ 。例如

$$P(x) \quad \text{包孕于} \quad P(a) \quad \sigma = \{a/x\}$$

$$P(x) \quad \text{包孕于} \quad P(a) \vee Q(z) \quad \sigma = \{a/x\}$$

$$P(x) \vee Q(a) \quad \text{包孕于} \quad P(f(a)) \vee Q(a) \vee R(y) \quad \sigma = \{f(a)/x\}$$

$$P(x) \vee Q(y) \quad \text{包孕于} \quad P(a) \vee Q(u) \vee R(w) \quad \sigma = \{a/x, u/y\}$$

对子句集来说，把其中包孕的子句删去后，不会影响该子句集的不可满足性。因此，可从子句集中删除那些包孕的子句。

3.4.5 归结演绎推理的归结策略

□ 限制策略

a) **支持集策略**：支持集策略是沃斯(Wos)等人在1965年提出的一种归结策略。

支持集策略是沃斯(Wos)等人在1965年提出的一种归结策略。

基本思想：要求每一次参加归结的两个亲本子句中，至少应该有一个是由目标公式的否定所得到的子句或它们的后裔。

可以证明**支持集策略是完备的**，即当子句集为不可满足时，则由支持集策略一定能够归结出一个空子句。

也可以把支持集策略看成是在广度优先策略中引入了某种限制条件，这种限制条件代表一种启发信息，因而有较高的效率

例设有如下子句集：

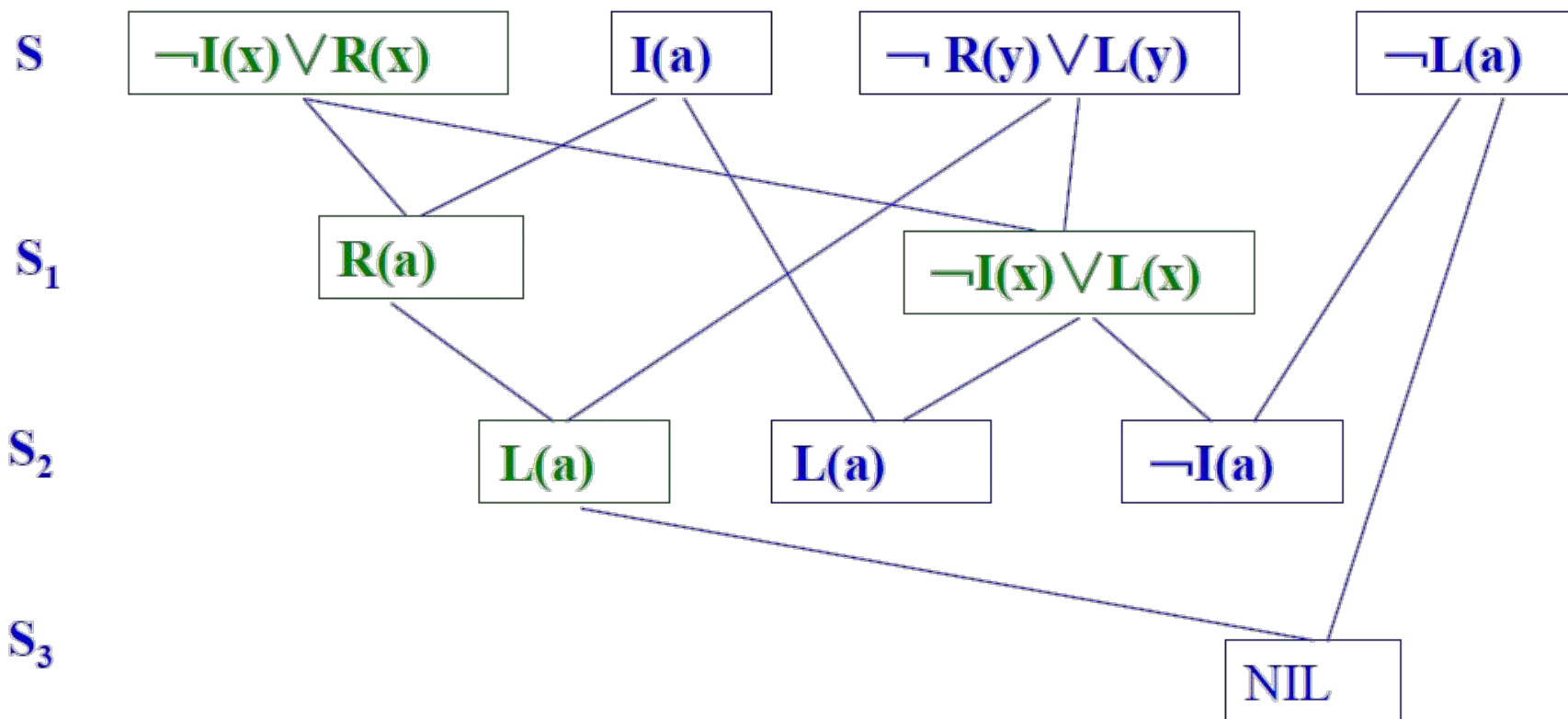
$$S = \{ \neg I(x) \vee R(x), I(a), \neg R(y) \vee L(y), \neg L(a) \}$$

其中， $\neg I(x) \vee R(x)$ 为目标公式的否定。用支持集策略证明S为不可满足。

3.4.5 归结演绎推理的归结策略

□ 限制策略

a) 支持集策略：支持集策略是沃斯(Wos)等人于1965年提出的一种归结策略。



可见，各级归结式数目要比广度优先策略生成的少，但在第二级还没有空子句。即这种策略限制了子句集元素的剧增，但却增加了空子句的深度。

3.4.5 归结演绎推理的归结策略

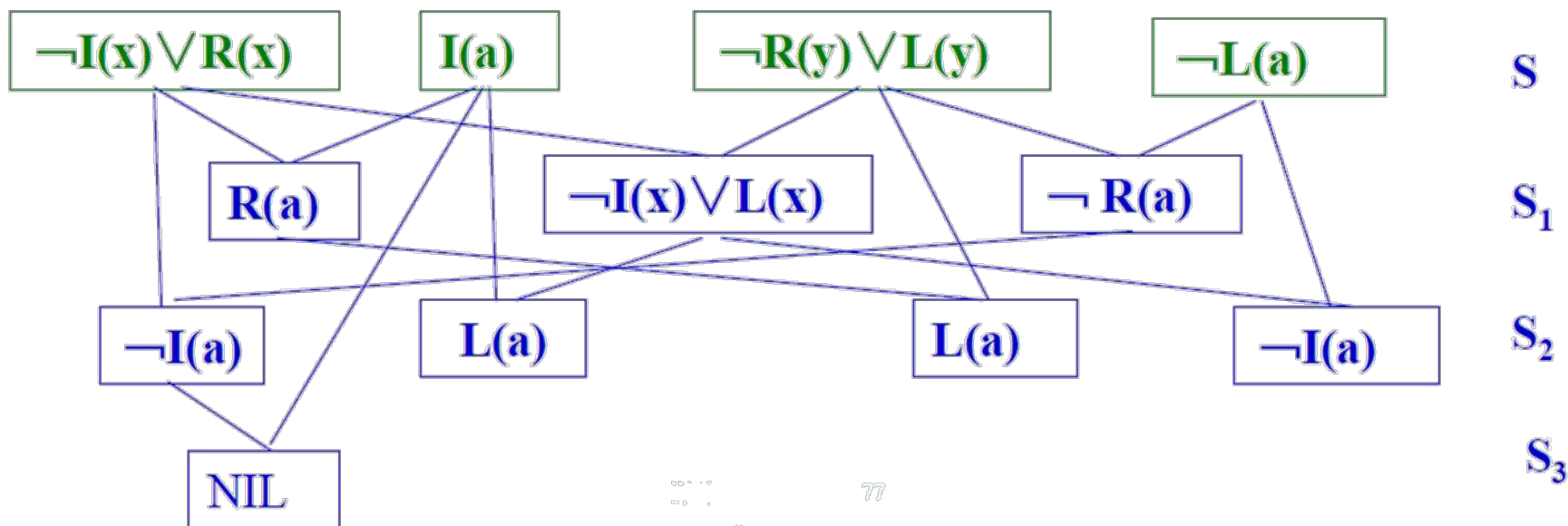
□ 限制策略

b) 线形输入策略

基本思想： 要求每次参加归结的两个亲本子句中，至少应该有一个是初始子句集中的子句。所谓初始子句集是指开始归结时所使用的子句集。

例 用线性输入策略证明如下子句集S为不可满足

$$S = \{ \neg I(x) \vee R(x), I(a), \neg R(y) \vee L(y), \neg L(a) \}$$



3.4.5 归结演绎推理的归结策略

□ 限制策略

b) 线形输入策略

该策略可限制生成归结式的数目，简单高效，但**是一种不完备策略**。如：

$$S=\{\neg Q(x) \vee \neg P(x), Q(y) \vee \neg P(y), \neg Q(w) \vee P(w), Q(a) \vee P(a)\}$$

从S出发可找到一棵归结反演树，但却不存在线性输入策略的归结反演树。

???

3.4.5 归结演绎推理的归结策略

□ 限制策略

c) 祖先过滤策略

这种策略与线性输入策略有点相似，但是，放宽了对子句的限制。

基本思想：每次参加归结的两个亲本子句，只要满足以下两个条件中的任意一个就可进行归结：

(1) 两个亲本子句中至少有一个是初始子句集中的子句。

(2) 如果两个亲本子句都不是初始子句集中的子句，则一个子句应该是另一个子句的先辈子句。

所谓一个子句(例如C1)是另一个子句(例如C2)的先辈子句是指C2是由C1与别的子句归结后得到的归结式。

可以证明祖先过滤策略也是完备的。

3.4.5 归结演绎推理的归结策略

□ 限制策略

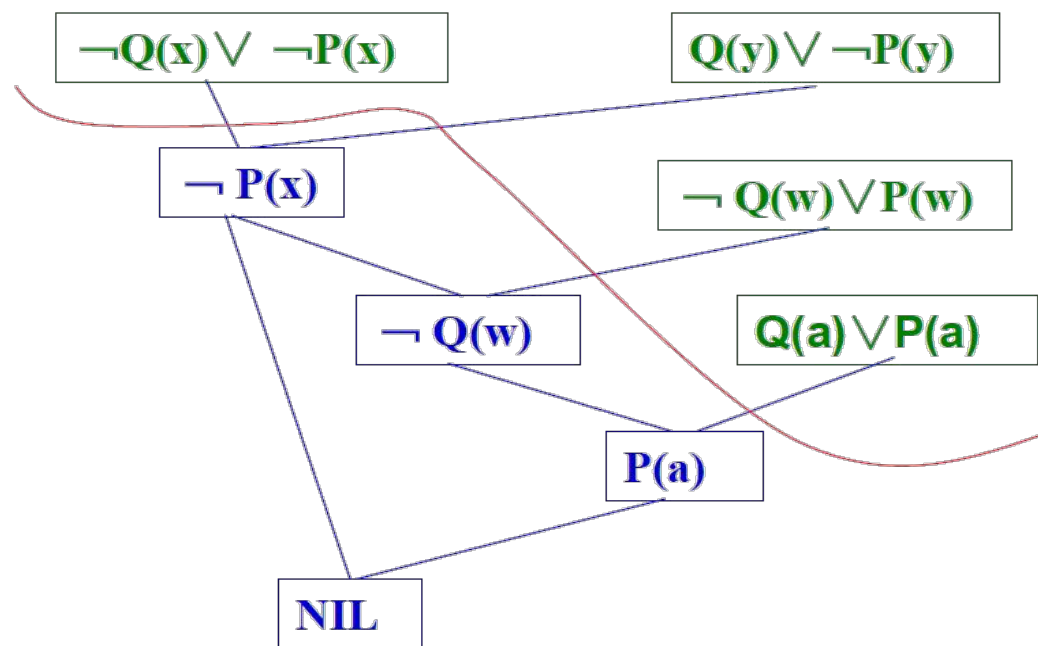
c) 祖先过滤策略

例 设有如下子句集:

$$S = \{\neg Q(x) \vee \neg P(x), Q(y) \vee \neg P(y), \neg Q(w) \vee P(w), Q(a) \vee P(a)\}$$

用祖先过滤策略证明S为不可满足

解: 其祖先过滤策略归结策略的归结过程如下图



3.4.5 归结演绎推理的归结策略

□ 限制策略

d) 单文字子句策略

单文字：如果一个子句只包含一个文字，则称此子句为单文字子句。

基本思想：要求每次参加归结的两个亲本子句中至少有一个子句是单文字子句。

单文字子句策略是对支持集策略的进一步改进，单文字子句策略是**不完备的**，即当子句集为不可满足时，用这种策略不一定能归结出空子句。

采用单文字子句策略，归结式包含的文字数将少于其亲本子句中的文字数，这将有利于向空子句的方向发展，因此会有较高的归结效率。

3.4.5 归结演绎推理的归结策略

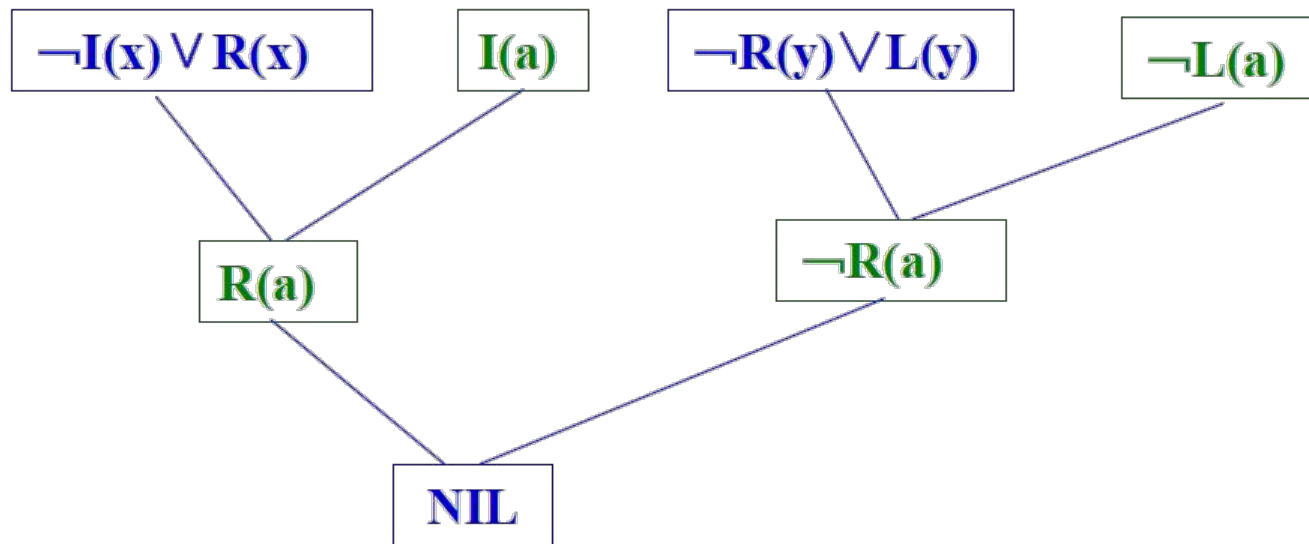
□ 限制策略

d) 单文字子句策略

例 设有如下子句集:

$$S = \{ \neg I(x) \vee R(x), I(a), \neg R(y) \vee L(y), \neg L(a) \}$$

用单文字子句策略证明S为不可满足。



小结：上面分别讨论了几种基本的归结策略，但在实际应用中，还可以把几种策略结合起来使用。总之，在选择归结反演策略时，主要应考虑其完备性和效率问题。

3.4.6 用归结反演求取问题的答案

归结原理除了可用于定理证明外，还可用来求取问题答案，其思想与定理证明相似。

一般步骤为：

- ①把已知条件用谓词公式表示，并化成相应的子句集 S_1 ；
- ②把待求解的问题也用谓词公式表示，然后将其否定，并与谓词ANSWER构成析取式 G_1 ；
- ③把 G_1 化为子句集 S_2 ，并把子句集 S_2 与 S_1 合并构成新子句集 S ；
- ④对子句集 S 应用谓词归结原理进行归结，在归结过程中通过合一置换，改变ANSWER中的变元；
- ⑤如果得到归结式ANSWER，则问题的答案就在ANSWER谓词中。

3.4.6 用归结反演求取问题的答案

例 已知:

- (1)如果 x 和 y 是同班同学, 则 x 的老师也是 y 的老师。
- (2)王先生是小李的老师。
- (3)小李和小张是同班同学。

问: 小张的老师是谁?

3.4.6 用归结反演求取问题的答案

解: $T(x, y)$ 表示 x 是 y 的老师, $C(x, y)$ 表示 x 与 y 是同班同学, 则

① 已知可表示成如下的谓词公式:

$$F_1: (\forall x)(\forall y)(\forall z)(C(x, y) \wedge T(z, x) \rightarrow T(z, y))$$

$$F_2: T(\text{Wang}, \text{Li})$$

$$F_3: C(\text{Li}, \text{Zhang})$$

将它们化成子句集为

$$S1 = \{ \neg C(x, y) \vee \neg T(z, x) \vee T(z, y), T(\text{Wang}, \text{Li}), C(\text{Li}, \text{Zhang}) \}$$

3.4.6 用归结反演求取问题的答案

② 把问题用谓词公式表示，并将其否定与谓词ANSWER作析取：

设小张的老师是u，则有 $T(u, \text{Zhang})$

$$G1: \neg T(u, \text{Zhang}) \vee \text{ANSWER}(u)$$

③ 将析取式G1化成子句集S2, 并将S1与S2合并为新子句集S:

$$S2 = \{ \neg T(u, \text{Zhang}) \vee \text{ANSWER}(u) \}$$

$$S = S1 \cup S2$$

$$= \{ \neg C(x, y) \vee \neg T(z, x) \vee T(z, y), \dots\dots\dots(a) \}$$

$$T(\text{Wang}, \text{Li}), \dots\dots\dots(b)$$

$$C(\text{Li}, \text{Zhang}), \dots\dots\dots(c)$$

$$\neg T(u, \text{Zhang}) \vee \text{ANSWER}(u) \dots\dots\dots(d)$$

3.4.6 用归结反演求取问题的答案

④ 应用归结原理进行归结：

(e) $\neg C(Li, y) \vee T(Wang, y)$ [a与b归结, (Li/x, Wang/z)]

(f) $\neg C(Li, Zhang) \vee ANSWER(u)$ [d与e归结, (Wang/u, Zhang/y)]

(g) $ANSWER(Wang)$ [c与f归结]

⑤ 得到归结式 $ANSWER(Wang)$ ，答案即在其中：

$u=Wang$ ，即小张的老师是王先生。